

本地 Qwen 与 Multi-Agent

从零开始：运行、追踪、逐行读代码与修改

项目：/home/xu

版本：详细教学版 · 2026-10-02（日本时间）

覆盖 26 个文件、586 行原始代码/配置/提示词，每行有解释。

先读第 1 章即可运行；第 5 章按真实文件追踪全过程；第 8—9 章给出代码替换与模型更新示例；附录按文件逐行查阅。

GPU 模型推理已经验证；电路结果是 Mock 合成数据，不是 SPICE。此次编写手册没有替换模型或执行 GPU 实验。

可编辑来源：docs/guide/chapters.md 与 docs/guide/make_annotations.py。完整可复制文字与代码：docs/USER_GUIDE.md。

目录

本地 Qwen 与 Multi-Agent：从零开始的完整使用手册	5
1. 先回答：我到底要运行哪个文件？	6
1.1 最短的可执行步骤	6
1.2 为什么要分成启动服务和运行实验？	6
1.3 哪些输出表示成功？	6
1.4 第二天重新登录后怎么做？	6
2. 零基础概念：这些东西分别是什么？	8
2.1 文件夹、文件、路径与当前目录	8
2.2 Python、解释器、库、函数与对象	8
2.3 变量、字典、列表与 JSON	8
2.4 Agent 在这个项目中的准确含义	9
2.5 API、HTTP、客户端和服务端	9
2.6 GPU、显存、CUDA、PyTorch 和 vLLM	9
3. 所有软件实际安装在哪里？	10
3.1 项目解释器与虚拟环境	10
3.2 运行依赖的精确位置	10
3.3 模型、缓存、日志与系统复用部分	11
4. 想修改什么，就去哪个文件？	12
5. 从第一条命令到结束：逐个文件追踪 workflow	13
5.1 阶段 A：准备终端环境	13
5.2 阶段 B：启动后台模型服务	13
5.3 阶段 C：一次可选的 API 冒烟测试	14
5.4 阶段 D：正式运行 Multi-Agent	14
5.5 每一次 Agent 调用内部，又发生了什么？	15
5.6 阶段 E：停止模型服务	16
6. 一次实际数据如何在文件和内存之间流动？	17
7. 不改 Python 也能完成的设置调整	18
7.1 改实验指标和迭代次数	18
7.2 改卡号、端口和上下文	18

7.3 改角色提示词	18
8. Python 层面如何修改：逐项给出原语句和替换语句	19
8.1 让 Agent 达标后直接停止，不依赖模型同意	19
8.2 把历史记录也传给优化 Agent	19
8.3 让每个角色使用不同 temperature	19
8.4 在仿真前增加一个 Review Agent	20
8.5 用确定性 Python 规则代替模型的优化建议	20
8.6 扩大参数范围时，改哪里？	21
8.7 修改 netlist 参数读取方式，避免依赖字典顺序	21
8.8 真正接入电路仿真器，需要改动哪些层？	22
8.9 如何检查你改过的 Python？	22
9. 如何更新模型，而不把几个“版本”混为一谈？	23
9.1 先区分四件事	23
9.2 同一个 Qwen3-8B 仓库，取得新的固定快照	23
9.3 从 Qwen3-8B 换成 Qwen3-4B 的完整配置更新示例	23
9.4 换成别的模型家族时，Python 哪些地方可能要改？	24
9.5 升级 vLLM 或 PyTorch，应该修改哪里？	24
9.6 模型切换失败如何回退？	25
10. 遇到问题，按文件定位	26
11. 如何阅读后面的逐行附录	27
附录 A：当前源文件逐行解释	28
A01. scripts/activate.sh	28
A02. scripts/env.sh	29
A03. scripts/start.sh	30
A04. scripts/service.py	31
A05. config/settings.json	36
A06. scripts/test_qwen.sh	36
A07. scripts/test_qwen.py	38
A08. scripts/run_demo.sh	38
A09. analog_agents/__init__.py	39
A10. analog_agents/config.py	40
A11. analog_agents/run.py	42
A12. analog_agents/agents.py	45

A13. analog_agents/client.py	47
A14. prompts/architecture.md	48
A15. prompts/sizing.md	48
A16. prompts/simulation.md	48
A17. prompts/optimization.md	49
A18. analog_agents/simulation.py	50
A19. scripts/stop.sh	52
A20. scripts/download_model.py	53
A21. scripts/install.sh	54
A22. scripts/test_agents.py	55
A23. tests/test_workflow.py	57
A24. pytest.ini	58
A25. scripts/audit_paths.py	59
A26. .venv/bin/activate	61
附录 B：校验与文档维护	67

本地 Qwen 与 Multi-Agent : 从零开始的完整使用手册

这份手册对应 /home/xu 中实际存在的项目。阅读对象是没有 Python、Agent、API、JSON 或 GPU 使用经验的人。你可以先读第 1—5 章运行系统，再读第 6—9 章理解机制，最后对照逐行附录学习代码。本文的代码修改示例是教学方案，没有自动改动现有 Agent 程序，也没有替换现有模型。

当前已验证的组合是 Python 3.11.13、PyTorch 2.6.0 + CUDA 12.4、vLLM 0.8.5 和 Qwen3-8B。模型加载、API、四个角色独立调用和端到端流程已经实测。电路部分仍然是 Mock，即人为写出的公式产生的合成数据，没有执行真实 SPICE，也没有工艺 PDK。

逐行讲解范围：所有项目自编的运行、启动、停止、下载、安装、测试与路径检查脚本，六个 analog_agents 模块、完整 settings.json、四份提示词，以及实际使用的 .venv/bin/activate 自动生成脚本。空行、注释、括号续行也单独标明。第三方 vLLM、PyTorch、Python 标准库和 NVIDIA 二进制库有海量内部代码，本手册说明其实际入口、位置、输入输出和作用，不声称把这些依赖源码逐行讲完。模型权重是数值文件，不是可逐行解释的 Python 代码。生成 PDF 的排版程序不参与 Agent 运行。

行号取自本次读取的源文件；代码以后有变化时应重新生成手册。附录不是重新编写的伪代码，而是把磁盘中的原始代码逐行摘出解释。PDF 中一条长代码可能视觉上折成几行，左侧 L 编号仍表示源文件的一行。复制修改示例应优先使用 Markdown 源文档，避免 PDF 自动换行破坏缩进。

1. 先回答：我到底要运行哪个文件？

1.1 最短的可执行步骤

在已经 SSH 登录服务器的终端中，一行一行输入以下命令。不要把这些命令输入 Python 的 `>>>` 提示符里；它们是交给 Bash 终端执行的。

```
cd /home/xu
source scripts/activate.sh
bash scripts/start.sh
bash scripts/test_qwen.sh
bash scripts/run_demo.sh
bash scripts/stop.sh
```

第一行进入项目目录；第二行准备当前终端的 Python 和缓存环境；第三行启动本地模型服务并等待 Ready；第四行检查模型是否能返回 $19+23=42$ ；第五行才是真正开始一次四角色 Multi-Agent 实验；最后一行在不再使用模型时停止后台服务。第四行是建议执行的验证步骤，不是 Agent 流程自身的一部分。

你日常真正需要记住的运行入口是 `/home/xu/scripts/run_demo.sh`。它最终启动的是 `/home/xu/analog_agents/run.py` 中的 `main()` 函数。它本身不会启动 Qwen，所以必须先执行 `start.sh`。你不能只双击 `run.py` 就期待整个系统自动准备好。

1.2 为什么要分成启动服务和运行实验？

Qwen3-8B 权重大约 16 GB。把这些数值从磁盘加载到 GPU 需要时间。`start.sh` 把模型加载一次，并让后台程序持续等待请求；`run_demo.sh` 是一个较小的前台程序，每次运行都向这个后台服务发送若干请求，保存一次实验，然后退出。你可以在模型服务仍运行时多次执行 `run_demo.sh`，不必每次重新加载权重。

`run_demo.sh` 结束并不等于 GPU 已释放。只有 `stop.sh` 停止服务后，模型占用的显存才会释放；`nvidia-smi` 的显示可能需要短暂等待才更新。关掉 Agent 客户端或运行 `deactivate` 都不会自动停止后台模型。

1.3 哪些输出表示成功？

`start.sh` 先打印 `Starting PID ...`，稍后打印 `Ready: http://127.0.0.1:8000/v1`。只看到 `Starting` 不代表加载完成。`test_qwen.sh` 成功时打印 `Local Qwen API: PASS` 和 `answer=42`。`run_demo.sh` 会打印 `architecture: OK`、`sizing: OK`、`simulation 0: MOCK ...`，最后给出 `Results: /home/xu/outputs/某个时间戳目录`。

`summary.json` 的 `status=completed` 仅表示程序正常完成了这一轮工作流程，不表示电路合格。是否满足合成目标，要查看 `final.simulation.checks` 与 `all_targets_met_synthetically`。`real_circuit_validated=false` 表示未验证真实电路。出现 `Traceback`、`failure.json` 或进程非零退出时，要按错误处理，不可把它当成实验成功。

1.4 第二天重新登录后怎么做？

```
cd /home/xu
source scripts/activate.sh
nvidia-smi
bash scripts/start.sh
bash scripts/run_demo.sh
bash scripts/stop.sh
```

重新登录会创建新的终端环境，通常要重新 source。系统已经安装，不要每天重新执行 install.sh 或下载模型。如果 start.sh 提示服务已经运行，先确认本项目服务状态；没有必要反复启动同一实例。不要使用 killall python 或 pkill 杀掉共享服务器上的其他任务。

2. 零基础概念：这些东西分别是什么？

2.1 文件夹、文件、路径与当前目录

/home/xu 是绝对路径，从 Linux 文件系统最上层 / 开始。scripts/run_demo.sh 是相对路径，只有当前目录是 /home/xu 时才指向本项目脚本。cd 改变当前目录；pwd 显示当前目录；ls 列出文件。以点开头的 .venv 和 .runtime 是隐藏目录，不是特殊的远程地址，用 ls -a 可以看到。

.sh 文件装着 Bash 终端指令；.py 文件装着 Python 程序；.json 文件保存有格式的数据；.md 文件保存可阅读的文字；.safetensors 保存模型权重数值；.cir 保存 SPICE 风格的电路连接描述。扩展名帮助人和工具区分用途，但并不保证内容正确。

2.2 Python、解释器、库、函数与对象

Python 是一种编程语言。解释器是执行这种语言的软件，在这里从 .venv/bin/python 启动。程序中的 import 表示使用其他模块已有的功能，不是重新安装软件。安装依赖发生在安装阶段；日常 import 只是从本地文件读取并执行模块初始化。

函数是一段有名字、可重复调用的步骤。例如 load_config() 读取配置并返回结果。def 定义函数时，其内部正文还不执行；遇到 main() 或 load_config() 这样的调用时才执行。括号中是传给函数的输入，return 是函数交回的结果。

class 定义一种对象的模板。Agent 是模板，Agent("sizing", client) 创建一个具体的尺寸设计角色对象。self 代表正在使用的那个对象；self.prompt 把该对象自己的提示词存起来。对象不是另一个模型，也不必对应另一个 Linux 进程。

2.3 变量、字典、列表与 JSON

变量是给一份数据起名字，例如 params 指向当前电路参数。Python 字典像一张按名称取值的表：

params["bias_ua"] 取偏置电流。列表是一串按顺序排列的值，例如 history 保存每次评估记录。列表下标从 0 开始，history[-1] 表示最后一条记录。

```
params = {"bias_ua": 40, "compensation_pf": 2}
current = params["bias_ua"]
history = []
history.append(params)
```

第一行建立字典；第二行读出 40；第三行建立空列表；第四行把 params 对应对象加入列表。这是 Python 代码，不是终端命令；可以保存到 .py 文件执行，不能直接当成 bash 命令逐行输入。

JSON 是一种把数据写成文本的通用格式。API 通信和输出文件都使用它。JSON 本身不会调用 GPU，也不会执行工作流程。JSON 字段名必须用双引号，布尔值写 true/false，不能写注释，最后一个字段后面不能留多余逗号。

```
{
  "bias_ua": 40,
  "synthetic": true,
  "analyses": ["op", "ac"]
}
```

Python 中相近的写法是 True/False；json.dumps 把 Python 字典转成 JSON 文本，json.loads 反过来把文本变成字典。Schema 是检查这份数据长什么样的规则，例如 bias_ua 必须是 1—300 之间的数字。Schema 校验能拒绝缺字段和越界参数，却不能证明模型的电路结论正确。

2.4 Agent 在这个项目中的准确含义

四个 Agent 是四套角色指令和输出格式，分别负责架构、初始尺寸、仿真计划、参数调整。它们都调用同一个 Qwen3-8B 服务，运行顺序由 run.py 的 Python 代码决定，并不是四个各自自由运行、自动相互聊天的大模型。

系统没有自动训练或更新模型权重，没有长期记忆数据库，也没有自动扫描所有项目文件。模型只看到 client.py 明确传给它的 system 和 user 消息。run.py 保存的 history 不会自动变成模型记忆，必须显式加入下一次请求才能让模型看到。

2.5 API、HTTP、客户端和服务端

API 是软件之间约定的“如何提出请求、如何返回结果”。客户端是提出请求的程序，这里是 client.py 加 OpenAI Python SDK。服务端是等待并处理请求的程序，这里是 vLLM 的 api_server.py。HTTP 是它们传递请求的通信协议。

127.0.0.1 是本地回环地址，在服务器上指服务器自己，在你的个人电脑上则指个人电脑自己。8000 是程序监听的端口号，相当于同一台机器上某个服务的入口编号。/v1/chat/completions 是具体接口路径。这个项目使用本机网络接口传消息，但不需要把设计请求发给 OpenAI 云端。

代码使用 from openai import OpenAI，是因为这个 SDK 支持 OpenAI 格式的接口。base_url 明确改成了本机 vLLM；api_key="local-only" 是本地占位字符串，不是真实云端密钥，也不意味着服务已经启用身份认证。同一服务器上能访问该回环端口的其他进程也可能调用它。

2.6 GPU、显存、CUDA、PyTorch 和 vLLM

CPU 执行普通控制逻辑，例如读取 JSON、保存文件和计算 Mock 公式。GPU 擅长并行计算大量数值，本项目用它运行 Qwen。显存是 GPU 自己的内存，用于放模型权重、推理中间结果和对话缓存；磁盘里有模型文件，不代表它已经进入显存。

NVIDIA 驱动是系统与 GPU 硬件沟通的软件，当前为 550.144.03。CUDA 运行库提供 GPU 运算所需的基础功能。PyTorch 是张量计算库，vLLM 在它及 CUDA 等组件之上负责高效加载和调用模型。Qwen 是模型架构加权重，不能仅靠一个 .safetensors 文件对外提供 API。

nvidia-smi 显示 CUDA 12.4，说明驱动所支持的 CUDA 版本范围，并不等于安装了同版完整开发工具包。当前 PyTorch 自带的运行库是 CUDA 12.4；系统 PATH 中 nvcc 是 10.1，本项目推理没有依靠这个旧编译器。不要为了本项目修改共享服务器驱动。

GPU=0 指 nvidia-smi 列出的第 0 张卡。脚本通过 CUDA_VISIBLE_DEVICES 让模型进程只看到这一张卡；在进程内部这张卡也编号为 cuda:0。这里四个 Agent 不需要四张卡。gpu_memory_utilization=0.5 是 vLLM 的显存预算比例，不是把 GPU 算力限制在 50%；实测推理时利用率可以达到 95%。

3. 所有软件实际安装在哪里？

3.1 项目解释器与虚拟环境

虚拟环境的作用是把本项目的 Python 软件包与系统 Python 分开。source scripts/activate.sh 会让当前终端优先找到 .venv/bin 下的 python 等命令，但没有修改 .bashrc 或系统 Python。下面每条路径都对应此次检查的实际安装位置。

项目	完整位置	作用
日常 Python 入口	/home/xu/.venv/bin/python	所有项目脚本实际选用的解释器入口
解释器真正文件	/home/xu/.runtime/python/cpython-3.11.13-linux-x86_64-gnu/bin/python3.11	上述入口解析到的 Python 3.11.13
Python 标准库	/home/xu/.runtime/python/cpython-3.11.13-linux-x86_64-gnu/lib/python3.11/	json、pathlib、subprocess 等内置模块
虚拟环境包目录	/home/xu/.venv/lib/python3.11/site-packages/	本项目安装的第三方库
uv 工具	/home/xu/.runtime/tools/bin/uv	安装解释器和依赖，日常推理不必运行
激活脚本	/home/xu/.venv/bin/activate	uv 自动生成，由 scripts/activate.sh source

.venv/bin/python 是指向项目内解释器的链接，而不是另一份完整 Python 拷贝。库安装在 .venv，基础解释器及标准库在 .runtime/python；两者合作完成隔离环境。不要移动其中一个文件夹后期待另一个仍能运行。

3.2 运行依赖的精确位置

下表的 P 只是为了让表格容易读，完整前缀为 /home/xu/.venv/lib/python3.11/site-packages。P/torch/ 就是 /home/xu/.venv/lib/python3.11/site-packages/torch/，不是需要你新建的 P 文件夹。

软件和版本	目录	什么时候使用
vLLM 0.8.5	P/vllm/	start.sh 启动后台模型服务；实际入口 P/vllm/entrypoints/openai/api_server.py
PyTorch 2.6.0+cu124	P/torch/	服务加载权重和执行 GPU 张量运算；原生库在 P/torch/lib/
Transformers 4.51.3	P/transformers/	模型配置、分词器及 Qwen 相关支持
Hugging Face Hub 0.30.2	P/huggingface_hub/	download_model.py 下载模型；服务可用本地缓存逻辑
OpenAI SDK 1.78.1	P/openai/	client.py 发送本地 HTTP 请求
jsonschema 4.26.0	P/jsonschema/	对模型返回字典及仿真参数作结构/范围检查
NVIDIA CUDA 运行库	P/nvidia/	GPU 运行依赖；libcudart 位于 cuda_runtime/lib/libcudart.so.12
Triton / xformers / xgrammar 等	P/triton/、P/xformers/、P/xgrammar/	vLLM 的计算或结构化输出依赖，由安装器一并解决
ReportLab 4.4.1	P/reportlab/	只在生成 PDF 时使用
pytest 8.3.5	P/pytest/ 和 P/_pytest/	只在运行离线测试时使用

P/nvidia/cublas/、 cudnn/、 nccl/ 等目录提供不同的 GPU 数学/通信库。你通常不需要直接调用它们，服务通过上层库自动加载。全部已安装依赖的版本清单在 requirements.lock ； requirements.txt 只列主要直接依赖。改这两个文本文件本身不会立即改变已安装软件。

3.3 模型、缓存、日志与系统复用部分

内容	路径	是否软件/数据
Qwen3-8B 权重	/home/xu/models/Qwen3-8B/model-00001-of-00005.safetensors 等五个分片	模型数值数据
权重索引	/home/xu/models/Qwen3-8B/model.safetensors.index.json	告诉加载器每个权重在哪个分片
模型配置	/home/xu/models/Qwen3-8B/config.json	模型层数等结构信息；不是项目 settings.json
分词文件	/home/xu/models/Qwen3-8B/tokenizer.json、tokenizer_config.json、 vocab.json、 merges.txt	把文字转换为 token 数字并套用对话模板
生成默认值	/home/xu/models/Qwen3-8B/generation_config.json	模型作者提供的默认生成参数，可能影响未显式设置项
缓存	/home/xu/.runtime/cache/	Hugging Face、 uv、 PyTorch、 Triton、 CUDA 等各自子目录
临时文件	/home/xu/.runtime/tmp/	下载、测试和运行的临时工作位置
运行记录	/home/xu/logs/	模型日志、服务 PID 信息和验证结果
实验产物	/home/xu/outputs/	每次实验一个带时间戳的子文件夹
Bash	/usr/bin/bash 或 /bin/bash	已存在的系统终端解释器，只读复用
NVIDIA 查询工具	/usr/bin/nvidia-smi	已存在的系统 GPU 状态工具
NVIDIA 驱动用户库	/lib/x86_64-linux-gnu/libcuda.so.1	系统驱动的一部分，只读复用；内核驱动也由系统管理
Git 2.25.1	/usr/bin/git	系统已有工具，当前流程不调用它

项目安装的软件、模型和缓存都放在 /home/xu 内，但不是所有硬件支撑软件都重新安装在这里： Bash、系统驱动和 Linux 本来就存在。无需也不应把系统 NVIDIA 驱动复制进项目。没有使用 Conda，也没有安装真实电路仿真器。

查看当前使用的解释器，使用下面命令；输出应指向本项目。 which 只查看命令来源， readlink 查看链接最终目标，都不会安装或修改软件。

```
cd /home/xu
source scripts/activate.sh
which python
readlink -f .venv/bin/python
python -m pip show torch vllm openai
```

4. 想修改什么，就去哪个文件？

修改目的	修改的项目文件	生效时间
GPU、端口、模型目录、显存比例、上下文长度	config/settings.json	停止旧服务并重新启动
实验指标、最大迭代次数、temperature、max_tokens、超时	config/settings.json	下一次启动 run_demo.sh 时读取
架构角色指令	prompts/architecture.md	下一次创建 Agent 时读取
尺寸角色指令	prompts/sizing.md	同上
仿真计划角色指令	prompts/simulation.md	同上
优化角色指令	prompts/optimization.md	同上
每个角色输出哪些字段、允许的数值范围	analog_agents/agents.py	下一次实验生效；同时更新使用这些字段的代码
请求发到哪里、消息如何组成、如何处理响应	analog_agents/client.py	下一次实验生效
谁先运行、谁后运行、循环次数与停止条件	analog_agents/run.py	下一次实验生效
网表连接、Mock 公式、仿真工具接入	analog_agents/simulation.py	下一次实验生效
配置合法性检查和根目录限制	analog_agents/config.py	下次加载配置时生效
vLLM 的启动参数与进程管理	scripts/service.py	下次重启模型服务生效
下载哪些模型文件	scripts/download_model.py	下次主动下载时生效
缓存/临时路径和环境变量	scripts/env.sh	新启动的脚本生效；已有后台进程不变
依赖版本与重装方式	requirements.txt、requirements.lock、scripts/install.sh	真正执行安装命令后生效

修改提示词不等于修改角色的程序逻辑。例如提示词写“最多优化十次”，而 run.py/config 中仍限制为 2，程序只会做至多两次参数更新。提示词要求输出新字段，但 agents.py 的 additionalProperties=false 不允许它，响应会被拒绝。模型回答“目标已达成”，也不能覆盖 Python 自己算出的 checks=false。

settings.json 与 models/Qwen3-8B/config.json 是两个完全不同的配置文件。前者由你维护，用于本项目运行；后者是模型仓库文件，描述神经网络架构。不要把 gpu、max_iterations 等项目设置塞进模型目录里的 config.json。

5. 从第一条命令到结束：逐个文件追踪 workflow



箭头表示主要控制/数据交接；后台模型服务持续运行，客户端每次请求通过 API 往返。

5.1 阶段 A：准备终端环境

你在终端输入 `source scripts/activate.sh`。Bash 读取 `/home/xu/scripts/activate.sh`；该文件先通过自己的所在目录定位并 `source /home/xu/scripts/env.sh`。`env.sh` 计算 `PROJECT_ROOT=/home/xu`，设置 `TMPDIR`、`HF_HOME` 等变量，确保临时文件与缓存留在项目内，并创建必要目录。

`activate.sh` 接着 `source /home/xu/.venv/bin/activate`。这个自动生成文件把 `.venv/bin` 放到 `PATH` 前面，设置 `VIRTUAL_ENV`，并给提示符加上环境名称。此时没有加载 Qwen、没有调用 GPU，也没有执行 Agent。`source` 的改变发生在当前终端；如果改用 `bash scripts/activate.sh`，变化只留在短命子进程中，回到原终端时不会完成同样的激活。

5.2 阶段 B：启动后台模型服务

B1. `bash scripts/start.sh` 让 Bash 读取 `/home/xu/scripts/start.sh`。它启用错误检查，重新 `source scripts/env.sh`，进入 `/home/xu`，然后用明确的 `.venv/bin/python` 执行 `scripts/service.py`，并把 `start` 作为命令行参数传入。

B2. Python 加载 `service.py` 的 `import`。引用 `analog_agents.config` 时，会先初始化 `analog_agents/__init__.py`，再执行 `config.py` 的顶层代码，确定 `ROOT`。`import` 不会自动执行 `config.py`

中 `load_config` 的函数正文。`service.py` 随后定义 `birth`、`owned`、`stop`、`start` 函数，直到走到文件末尾 `__main__` 分支。

B3. `service.py` 打开 `logs/service.lock`，用 `fcntl.flock` 取得非阻塞排他锁。它根据 `sys.argv[1]` 的 `start` 选择执行 `start()`。这把锁防止同一项目的启动/停止管理动作同时修改状态文件；它不是实验室 GPU 资源预约，也不会一直锁住整个后台服务寿命。

B4. `start()` 调用 `config.py` 的 `load_config()`，读取 `/home/xu/config/settings.json`，解析 JSON，检查 GPU 编号、端口、显存比例、上下文长度、迭代次数与 mock 后端是否合法。它查询 `logs/service.json`（若存在），避免重复启动自己仍在运行的服务；还尝试绑定 `127.0.0.1:8000`，检查端口是否空闲。

B5. `start()` 调用系统 `/usr/bin/nvidia-smi`，查询所选 GPU 的 UUID、已用显存、总显存和利用率。已用显存大于 1024 MiB 或利用率大于 5% 就抛错退出；没有自动找另一张卡，更不会杀掉占用者。空闲检查与真正占用之间仍可能有别的任务进入，因此共享环境应遵守实验室调度规则。

B6. 它把 `settings.json` 中 `model_dir` 解析成 `/home/xu/models/Qwen3-8B`，先检查模型 `config.json` 是否存在。这一检查不是对五个权重分片的完整性验证；完整模型能否加载，后面的 vLLM 启动才会真正验证。模型缺失时，`start.sh` 不会自动下载。

B7. `service.py` 复制当前环境变量，为子进程设置 `CUDA_VISIBLE_DEVICES="0"`、`CUDA_DEVICE_ORDER="PCI_BUS_ID"`、`VLLM_USE_V1="0"` 和离线模式。然后通过 `subprocess.Popen` 启动新的后台进程，实际命令入口是 `.venv/bin/python -m vllm.entrypoints.openai.api_server`。

B8. Python 按模块名找到 `/home/xu/.venv/lib/python3.11/site-packages/vllm/entrypoints/openai/api_server.py`。从这里进入第三方库内部：vLLM 解析 `--model`、`--host` 等参数，读取模型目录下的配置、分词文件、权重索引和五个 `safetensors` 分片，在 GPU 0 分配显存、加载权重并预热。它通过 PyTorch、CUDA 运行库和系统驱动完成 GPU 运算。这里描述逻辑依赖，不假定库内部并行加载文件的精确先后顺序。

B9. vLLM 监听 `127.0.0.1:8000`，所有标准输出和报错追加到 `logs/qwen.log`。`service.py` 将子进程 PID、进程出生标记、GPU UUID、端口和命令写到 `logs/service.json`，再反复请求 `/health`。健康检查返回 HTTP 200 才打印 Ready 并结束启动管理程序。后台模型进程因为 `start_new_session=True` 仍然继续运行。

5.3 阶段 C：一次可选的 API 冒烟测试

`bash scripts/test_qwen.sh -> source scripts/env.sh -> .venv/bin/python scripts/test_qwen.py`。`test_qwen.py` 调用 `load_config`，创建 `LocalClient`，让模型回答 `19+23`，并用只允许 `answer` 整数的 Schema 约束输出。返回值通过校验且 `answer==42` 后，写 `logs/api_test.json`。这个测试调用的仍是本地 GPU 模型，不是把固定答案假装成模型响应。

5.4 阶段 D：正式运行 Multi-Agent

D1. `bash scripts/run_demo.sh` 读取同名脚本，准备环境并进入项目目录，然后执行 `.venv/bin/python -m analog_agents.run`。`-m` 表示“按照模块名执行”，会正确处理 `run.py` 中 `from .config` 等相对导入；直接 `python analog_agents/run.py` 可能报相对导入错误。

D2. Python 导入 `analog_agents/__init__.py` 与 `run.py` 依赖的 `config.py`、`client.py`、`agents.py`、`simulation.py`。这些导入建立函数、类和 Schema；在这个阶段只定义 Agent，不会自动发起四次模型请求。`run.py` 最后检测 `__name__=="__main__"`，然后调用 `main()`，正式进入实验。

D3. `main()` 再次读取 `config/settings.json`。它生成日本时区的时间戳，例如 `outputs/20261002-131535-070986/`，建立新实验目录，并把本次配置写成该目录下的 `config.json`。这个输出配置是快照，不是下一次实验的输入；真正的输入仍在项目 `config/settings.json`。

D4. `main()` 按 `architecture`、`sizing`、`simulation`、`optimization` 建立四个 Agent。每创建一个 Agent，就先创建一个 `LocalClient(cfg)`，再由 `Agent.__init__` 读取 `prompts/相应角色.md`。因此会有四个 Python 客户端对象，但都指向同一个模型服务，不会加载四份权重。Agent 把提示词读入内存一次，本次运行中途修改 `.md` 通常不会改变已经创建的 Agent。

D5. `main()` 建立 `context={"specification": ...}` 和 `history=[]`。`context` 是准备给角色看的材料；`history` 是本地累计的评估记录。最先运行 `architecture`，它收到设计指标。架构结果写入 `context["architecture"]` 和 `architecture.json`。接着 `sizing` 收到包含指标和架构结果的 `context`，返回 `parameters` 和 `rationale`，写入 `sizing.json`；从中取出 `parameters` 作为当前 `params`。

D6. 进入 `for step in range(max_iterations+1)`。默认 `max_iterations=2`，所以 `step` 可能依次是 0、1、2。`step=0` 是初始尺寸评估，后两次是允许的参数更新后评估。因此最多三次仿真评估，而不是“总共两次评估”。

D7. 每轮建立 `iteration-0` 等目录。先调用 `simulation` Agent，让它看到设计指标、初始架构/尺寸记录，以及顶层 `parameters` 中的当前尺寸。它返回 `analyses` 列表和理由。这里顶层 `parameters` 才是迭代后的当前值；`context["sizing"]` 仍保留初始设计，程序没有同步覆盖它。

D8. Python 直接调用 `simulation.py` 中 `MockSimulation().run(...)`。它校验当前参数，调用 `netlist()` 从固定模板生成 `conceptual_not_executed.cir`，再用写死的数学公式计算 `metrics`，逐项与目标比较得到 `checks`。它没有读取这个 `.cir` 去运行 SPICE，也没有调用 `ngspice`，且这些计算使用 CPU。`analyses` 的 `op/ac/tran` 会影响网表中追加哪些分析语句，但当前 Mock 指标仍全部计算，不是分别真正执行这些分析。

D9. `run.py` 把当前 `step`、`parameters`、`simulation_plan` 和 `simulation` 组装为 `record`，加入 `history`，并覆盖保存完整 `history.json`。覆盖是为了让文件始终包含当前所有已完成记录，不是只保存最后一轮。由于 `record` 是字典对象，后续给它增加 `optimization` 字段，`history` 中同一个对象也会出现这个字段。

D10. 如果 `step` 已达到最大迭代次数，立即 `break` 跳出循环；这一轮不会再调用 `optimization` Agent。如果未达到，就让 `optimization` Agent 看到 `context` 加当前 `record`：指标、架构、初始尺寸、当前参数、当前 Mock 结果和模拟公式。完整 `history` 默认没有传入。

D11. 把优化回答记录到 `record["optimization"]` 并再次保存 `history.json`。如果 `advice["stop"]` 为真且程序计算的 `all_targets_met_synthetically` 也为真，停止循环。两个条件必须同时成立；模型说停，但程序发现未达标时，不会因这条条件停下。

D12. 没有提前停止时，`params = advice["parameters"]` 把优化建议完整替换为下一轮当前尺寸，然后返回 D7。程序没有自动保留最优方案、没有回滚变差的建议，也没有保证每次建议都不同；同样参数也可能再次评估。

D13. 循环正常结束，保存 `summary.json`，里面 `final=history[-1]`。打印 `Results` 路径后 Python 进程退出。Qwen 后台服务仍在。`try` 内发生异常时写 `failure.json`，再 `raise` 把错误继续向终端抛出；但 `main()` 开头读取配置、建目录和建 Agent 位于 `try` 之前，这些早期错误可能只出现终端 `Traceback`，而没有 `failure.json`。

5.5 每一次 Agent 调用内部，又发生了什么？

`Agent.run(context) -> LocalClient.ask(prompt, context, SCHEMAS[name]) -> SDK chat.completions.create(...)`。`client.py` 建立两条消息：`system` 消息是角色提示词加输出 Schema，`user` 消息是 `context` 转成的 JSON 字符串。`temperature`、`max_tokens` 等来自 `settings.json`，`enable_thinking=False` 关闭 Qwen3 思考输出，`guided_json` 把结构要求发给 vLLM。

SDK 把请求发送到 `http://127.0.0.1:8000/v1/chat/completions`。vLLM 用分词器把文字变成 token ID，把它交给已经驻留 GPU 的 Qwen，逐步生成输出 token，再变回文本，并封装 HTTP JSON 响应。请求里不包含任意自动读取的磁盘文件，只有代码显式提供的消息。

client.py 取 response.choices[0]。若 finish_reason 不是 stop，例如达到 token 上限导致 length，就报“输出不完整”。正常时将 message.content 从 JSON 文本解析成 Python 字典，再用 jsonschema.validate 作结构和范围检查。通过后才 return 给 run.py。finish_reason="stop" 是这一次文字生成的结束原因，和 optimization 输出字段 stop=true 是两件不同的事。

一个 request 会等待模型返回后，Python 才执行下一步。当前没有 async、线程池或四角色并发。SDK 的 max_retries=1 表示某些可重试的请求错误会自动再尝试一次；timeout_seconds 是 SDK 网络超时设置，不能理解为整个实验严格在 180 秒内完成。

5.6 阶段 E：停止模型服务

bash scripts/stop.sh -> source env.sh -> .venv/bin/python scripts/service.py stop。service.py 读取 logs/service.json，借助 /proc/进程号/stat 比较进程出生标记，避免只按可能被复用的 PID 杀进程。确认属于已记录进程后，用 os.killpg 向该进程组发 SIGTERM，请它正常退出。

脚本最多轮询约 30 秒，检测进程退出或进入僵尸状态，然后删除服务状态文件；日志文件和模型权重不会删除。CUDA 显存释放可能在 nvidia-smi 中稍后才显示。脚本不会执行卸载、不清除 outputs，也不会替你退出虚拟环境。如果想取消当前终端的环境激活，另输入 deactivate。

6. 一次实际数据如何在文件和内存之间流动？

下面的示例来自已有实验 `outputs/20261002-131535-070986/`，不是本次编写手册重新运行的结果。初始参数 `input_w_um=20`、`input_l_um=1`、`bias_ua=40`、`compensation_pf=2`。Mock 初始增益为 60.612 dB、带宽 16 MHz、相位裕度 68 度、功耗 0.216 mW。

优化返回 `input_w_um=25`、`bias_ua=35`，其他字段保持。run.py 中 `params = advice["parameters"]` 让下一次仿真真正使用这些新值，于是 Mock 增益变为 61.775 dB、带宽 14 MHz、功耗 0.189 mW。目标增益是 65 dB，仍未达到，所以虽然 `summary.status` 是 `completed`，`final.simulation.checks.gain_db` 仍是 `false`。

这说明软件完成了“取建议、应用、重新计算”的闭环，不说明电路完成了优化。当前只保存最后一次评估作为 `final`，并没有挑选历史最优。如果未来某次建议使结果更差，也会照样保存在最后一轮；第 8 章说明如何修改这些逻辑。

文件	谁写入	什么时候看
<code>config.json</code> （实验目录内）	run.py 的 <code>save</code>	确认这次实验用了什么设置
<code>architecture.json</code>	run.py	查看架构角色原始回答
<code>sizing.json</code>	run.py	查看初始参数与理由，不是最终参数
<code>history.json</code>	run.py，每轮至少写一次	查看所有已完成评估和优化建议
<code>iteration-N/conceptual_not_executed.cir</code>	simulation.py	检查第 N 轮概念连接和数值；未执行
<code>summary.json</code>	run.py，正常结束后	查看最终评估及 workflow 完成状态
<code>failure.json</code>	run.py 的异常分支	查看 <code>try</code> 内失败原因；不是所有早期失败都能写入
<code>logs/qwen.log</code>	后台 vLLM 进程	查模型加载、API、显存和服务错误

不要改 `outputs` 内的 JSON 期待影响下一次运行：那是历史记录。修改输入请改 `config/settings.json`、`prompts` 或 `analog_agents` 中的代码。`history.json` 中出现一个字段，并不意味着它自动参与下一次请求；要看 `client.ask` 的 `payload` 究竟来自哪个字典。

7. 不改 Python 也能完成的设置调整

先备份要改的文件，再编辑。在下面示例中，备份保存在项目 docs/ 下。文件名用不同版本避免覆盖你想保留的旧备份。

```
cd /home/xu
cp config/settings.json docs/settings.before-edit.json
nano config/settings.json
```

nano 是终端文本编辑器；若系统没有安装，可用你已经使用的 VS Code Remote 编辑同一文件。保存后运行以下命令检查 JSON 语法；这只检查是否能解析，不会启动模型。

```
source scripts/activate.sh
python -m json.tool config/settings.json
python -c 'from analog_agents.config import load_config; print(load_config())'
```

7.1 改实验指标和迭代次数

把 settings.json 中的 "max_iterations": 2 替换为 "max_iterations": 4，就允许最多五次评估和四次优化建议。config.py 当前限制为 1—10，超过 10 需要同时改校验，不是改 JSON 一个数字就能生效。把 specification 中 "gain_db_min": 65 替换为 "gain_db_min": 60，会改变 Python 判断目标达成的门槛，而不是改变 Mock 公式。

把 "temperature": 0.2 改成 0.1 可减少生成的随机性，但不保证每次完全相同。把 "max_tokens": 1200 改成 1800，允许单次模型回答更长；输入加输出仍要放进 context_length，回答越长也可能越慢。

7.2 改卡号、端口和上下文

例如先确认 GPU 1 空闲，再把 "gpu": 0 替换为 "gpu": 1。把 "port": 8000 替换为 "port": 8001，会同时影响下次启动的服务和下次创建的 LocalClient，因为两者都读同一份配置。先 stop.sh，再修改，再 start.sh，避免后台仍在旧端口而客户端已经访问新端口。

context_length=8192 是输入与输出总 token 容量；max_tokens=1200 是单次输出上限。二者不同。gpu_memory_utilization=0.5 不要随意改到 1：当前校验最多允许 0.6，且共享服务器不应无确认占满设备。

7.3 改角色提示词

prompts/ 中每个文件是一段 system 指令，名字要与 agents.py/run.py 中的角色名称对应。可以把尺寸角色要求改为“说明每项参数的假设”，但仍要遵守规定 JSON 字段。改提示词不需要重启 Qwen；重新运行 run_demo.sh 会创建新 Agent 并读取新文本。

8. Python 层面如何修改：逐项给出原语句和替换语句

本章示例以当前源码为起点，彼此独立。不要不加判断把所有示例同时叠加。除非明确标为完整替换，代码块只是指定位置的一段，应保留原有缩进。四个空格表示一级 Python 代码块，缩进不同会改变逻辑或引起错误。示例没有自动应用到生产文件；后面的示例验证记录会说明检查到什么程度。

8.1 让 Agent 达标后直接停止，不依赖模型同意

当前 run.py 在得到 optimization 回答后判断：

```
if advice["stop"] and result["all_targets_met_synthetically"]:
    break
```

如果你的目标是“只要当前评估已满足所有合成指标，就连优化请求都不再发送”，应将下面这一段插入到 if step == cfg["max_iterations"]: 之前：

```
if result["all_targets_met_synthetically"]:
    break
```

原来的双条件判断可以保留，它在这个改法中对于达标场景已经不会再执行。新增判断的位置很重要：如果放在调用 optimization 之后，只能省掉下一轮评估，不能省掉这次模型请求。此修改依然只判断 Mock 指标，不会把 Mock 变为真实电路验证。

8.2 把历史记录也传给优化 Agent

当前 run.py 的原语句是：

```
advice = agents["optimization"].run(**context, **record)
```

替换为：

```
advice = agents["optimization"].run({
    **context,
    **record,
    "previous_iterations": history[:-1],
})
```

context 表示把 context 的键值展开到新字典；后面的同名键会覆盖前面的值。history[:-1] 是不包含本轮的旧记录列表，避免把本轮内容重复发送。这个字典马上被 json.dumps 转成文本发给模型，不会形成无限循环引用。更多历史会占用更多 token，可能要减少记录内容或调整上下文，而不是无限堆积。

8.3 让每个角色使用不同 temperature

第一步，在 settings.json 的 "temperature": 0.2 后添加一个字段，记得逗号：

```
"agent_temperatures": {
  "architecture": 0.3,
  "sizing": 0.1,
  "simulation": 0.1,
  "optimization": 0.1
},
```

第二步，在 agents.py 中找到 def run(self, context): 及其下一行，完整替换这个方法：

```
def run(self, context):
    temp = self.client.config.get("agent_temperatures", {}).get(
        self.name, self.client.config["temperature"])
    return self.client.ask(
        self.prompt, context, SCHEMAS[self.name], temperature=temp)
```

第三步，在 client.py 中把函数头 def ask(self, prompt, payload, schema): 替换为：

```
def ask(self, prompt, payload, schema, temperature=None):
```

同一函数内，把 temperature=self.config["temperature"], max_tokens=self.config["max_tokens"], 这一整行替换为：

```
temperature=(self.config["temperature"] if temperature is None else temperature),
max_tokens=self.config["max_tokens"],
```

.get(key, default) 表示字段不存在就使用默认值，这样 test_qwen.py 仍然可以不传角色温度调用 ask。None 表示“没有指定”，而不是 0；温度 0 是一个有效的明确值，不能用 if temperature 误判为未提供。这三处必须配套，否则可能出现“ask 不接受 temperature 参数”的错误。

8.4 在仿真前增加一个 Review Agent

这不是简单加一份 prompt 就能运行；必须增加输出 Schema、创建对象并在流程里调用它。先创建 prompts/review.md，内容如下：

```
You are a review agent. Check the proposed parameters against the given
specification and identify conceptual issues. No PDK is available.
Return approved (boolean) and reason (string) as JSON.
Do not claim real circuit validation. Be concise in Chinese.
```

在 agents.py 的 SCHEMAS 字典内，给 optimization 所在字段末尾加逗号，再加入：

```
"review": obj({"approved": {"type": "boolean"}, "reason": TEXT}),
```

在 run.py 中，把创建 agents 的原语句替换为：

```
agents = {
    name: Agent(name, LocalClient(cfg))
    for name in ("architecture", "sizing", "simulation", "optimization", "review")
}
```

在 run.py 循环中 step_dir.mkdir() 之后、plan = agents["simulation"].run(...) 之前，插入：

```
review = agents["review"].run(**context, "parameters": params)
save(f"review-{step}.json", review)
if not review["approved"]:
    raise ValueError("Review rejected: " + review["reason"])
```

这样每轮当前参数都会先经过 review，拒绝时触发已有 failure.json 路径，不会继续做本轮仿真。approved 是模型意见，仍不是电路安全证明。test_agents.py 的固定四角色列表不会自动扩展；若想五色独立测试，还应把该脚本列表补上 review。本例只是在“仿真前”插入一关，不能把 sizing 放到 architecture 之前，因为 sizing 本来依赖架构结果。

8.5 用确定性 Python 规则代替模型的优化建议

如果先想学清楚“改一个参数如何影响后面”，可以暂时用一条可理解的规则替换优化模型调用。只替换 run.py 中 advice = agents["optimization"].run(**context, **record) 这一行，换成下面整段：

```

next_params = dict(params)
needed_ratio = 10 ** ((cfg["specification"]["gain_db_min"] - 45) / 12)
next_params["input_w_um"] = min(
    200.0,
    max(params["input_w_um"], needed_ratio * params["input_l_um"] * 1.02),
)
advice = {
    "parameters": next_params,
    "rationale": "Synthetic formula rule; not a physical circuit optimizer.",
    "stop": result["all_targets_met_synthetically"],
}

```

`dict(params)` 建一个独立字典，避免直接改掉历史记录中旧参数；`needed_ratio` 是把 Mock 增益公式反解后的最小 W/L；乘 1.02 留少许数值余量；`min(200.0, ...)` 保持当前 Schema 中输入管宽度上限。其他参数沿用当前值，之后原代码会记录 `advice`、更新 `params`、重新计算。

这个例子不再让 Optimization Agent 调模型，虽然原来的对象仍被创建。它只针对现有 Mock 增益公式，不能用于真实晶体管设计，也不能保证所有目标或所有输入都可达。它用于理解 Python 控制流程与“模型建议”可以替换成不同算法。

8.6 扩大参数范围时，改哪里？

`agents.py` 的 `PARAMETERS` 决定模型尺寸和优化输出允许的范围。原字段：

```
"input_w_um": {"type": "number", "minimum": 1, "maximum": 200},
```

例如在你确认实验需要后，可替换为：

```
"input_w_um": {"type": "number", "minimum": 1, "maximum": 300},
```

`sizing` 和 `optimization` 的 Schema 都引用同一个 `PARAMETERS`，所以它们会一起使用新上限；`simulation.py` 的 `validate` 也引用它。但如果你同时采用 8.5 的规则，那里 `min(200.0, ...)` 还会额外限制为 200，也要有意修改成 300.0。JSON Schema 的修改不会自动寻找并替换其他文件里的数字。

新增参数比改上限更复杂：不仅要加 Schema，还要决定提示词、网表、公式和使用方如何读取。当前 `netlist()` 用 `PARAMETERS` 字段顺序解包成六个变量，贸然增加第七个字段会发生“解包值过多”。下一节给出先消除这一隐式依赖的改法。

8.7 修改 netlist 参数读取方式，避免依赖字典顺序

在 `simulation.py` 中，把下面的原语句：

```
w, l, load, stage, bias, cc = [p[k] for k in PARAMETERS["properties"]]
```

替换为：

```

w = p["input_w_um"]
l = p["input_l_um"]
load = p["load_w_um"]
stage = p["stage2_w_um"]
bias = p["bias_ua"]
cc = p["compensation_pf"]

```

原代码依赖 `PARAMETERS` 中六个字段的插入顺序；改成按名字读取更容易理解，也不会因为字典重排把晶体管宽度误当成电流。这仍然没有实现任何新的电路参数，只是先把参数与网表的对应关系写得明确。

8.8 真正接入电路仿真器，需要改动哪些层？

当前 config.py 明确 `assert simulation_backend == "mock"`，run.py 又直接写 `MockSimulation().run(...)`。因此单把 JSON 改成 "ngspice" 会立即失败；把 `assert` 删除而不改 run.py，则仍在跑 Mock，不能这样冒充接入成功。

可以先做一个诚实的工厂函数，为未来真实实现留入口。在 simulation.py 文件末尾新增：

```
def make_simulator(backend):
    if backend == "mock":
        return MockSimulation()
    raise NotImplementedError(
        "This backend has no validated simulator implementation yet: " + backend)
```

在 run.py 中，把 `from .simulation import MockSimulation` 替换为 `from .simulation import make_simulator`；把 `result = MockSimulation().run(params, cfg["specification"], plan["analyses"], step_dir)` 替换为：

```
result = make_simulator(cfg["simulation_backend"]).run(
    params, cfg["specification"], plan["analyses"], step_dir)
```

这一步保持原有 mock 限制和结果，不会自动安装仿真器。未来必须实现真实类：受控生成带合适模型的网表；用固定可执行文件、`shell=False` 和 `timeout` 运行仿真器；检查退出码、输出文件、收敛情况；解析带单位的数值；与指标比较；只有成功才返回 `real_spice_executed=True`。失败必须报错，不能偷偷用 Mock 数字补齐。

本项目没有 PDK、真实仿真器安装路径或经过验证的指标提取脚本，所以此处不能给出一条“把 X 换成 Y 就完成真实电路仿真”的虚假承诺。替换工具调用、物理模型和结果解析是不同工作，完成后还要有真实仿真测试。现有 LEVEL=1 概念网表不能代替工艺模型。

8.9 如何检查你改过的 Python？

先做语法检查，不启动 GPU、不导入你修改的业务模块：

```
cd /home/xu
source scripts/activate.sh
python - <<'PY'
import ast
from pathlib import Path
for folder in ("analog_agents", "scripts"):
    for path in Path(folder).glob("*.py"):
        ast.parse(path.read_text(), filename=str(path))
        print("syntax OK", path)
PY
python -m pytest -q --basetemp="$TMPDIR/pytest"
```

`ast.parse` 能发现缺冒号、括号不配对或缩进语法问题，但不能保证算法正确。现有四项测试覆盖参数拒绝、Mock 标签等，不会替你自动验证新角色。之后再启动服务、执行 `test_qwen.sh`、`test_agents.py`、`run_demo.sh`，并检查新 outputs。文档示例通过语法检查不等于已经做过真实 GPU 集成测试。

9. 如何更新模型，而不把几个“版本”混为一谈？

9.1 先区分四件事

模型仓库 `model_repo` 是模型名字，例如 `Qwen/Qwen3-8B`；`model_revision` 是仓库内某一快照的 `commit`；`model_dir` 是下载到本机的目录；`served_model` 是 API 对外报出的别名。它们不是 `vLLM` 软件版本，也不是 Python 版本。

更新同一仓库权重快照、从 8B 换成 4B、升级 `vLLM` 推理框架，是三种不同操作。改 `served_model` 只改接口名字，不会使 GPU 自动加载另一个模型。服务使用 `--model` 指定的本地目录；离线模式下它不会自动替你追踪远程最新版本。

9.2 同一个 Qwen3-8B 仓库，取得新的固定快照

先停止旧服务并备份配置。下面命令只作为你日后主动更新时的完整做法，本次手册重写没有运行这个下载或替换操作。获取 `main` 的当前 SHA 需要联网；如果 `main` 仍是原来的 SHA，就没有新快照，不需要反复下载。

```
cd /home/xu
source scripts/activate.sh
bash scripts/stop.sh
cp config/settings.json docs/settings.before-model-update.json
python - <<'PY'
import json
from pathlib import Path
from huggingface_hub import HfApi
path = Path("config/settings.json")
cfg = json.loads(path.read_text())
new_sha = HfApi().model_info(cfg["model_repo"], revision="main").sha
if new_sha == cfg["model_revision"]:
    print("Already using this revision:", new_sha)
else:
    cfg["model_revision"] = new_sha
    cfg["model_dir"] = "models/Qwen3-8B-" + new_sha[:12]
    path.write_text(json.dumps(cfg, ensure_ascii=False, indent=2) + "\n")
    print("New pinned revision:", new_sha)
PY
python scripts/download_model.py
bash scripts/start.sh
bash scripts/test_qwen.sh
python scripts/test_agents.py
bash scripts/run_demo.sh
```

`HfApi().model_info` 只查询仓库元信息，`.sha` 取当前 `main` 指向的固定提交值；真正下载由下一条 `download_model.py` 完成。`cfg["model_dir"]` 改为新目录，避免把不同快照的文件混放，也保留旧版本用于回退。字符串 `[:12]` 取提交号前十二个字符作为目录后缀，完整 `revision` 仍保存完整提交号。

同一仓库 `revision` 也不保证永远与旧软件完全兼容，仍需实际加载验证。新目录可能再占约一份模型的磁盘空间，应先 `df -h /home/xu`。本机磁盘是共享资源，不要按首次部署的空闲量假定以后一直足够。

9.3 从 Qwen3-8B 换成 Qwen3-4B 的完整配置更新示例

这是可选的较小同系列模型示例，不是“最新模型”推荐，也没有在此服务器重新下载测试。Qwen 官方模型卡说明 Qwen3-4B 支持 Transformers 4.51+、`vLLM` 0.8.5+；最终能否在你当前环境工作仍以实际测试为准。参考：<https://huggingface.co/Qwen/Qwen3-4B>。


```

cd /home/xu
source scripts/activate.sh
bash scripts/stop.sh
cp config/settings.json docs/settings.before-4b.json
python - <<'PY'
import json
from pathlib import Path
from huggingface_hub import HfApi
path = Path("config/settings.json")
cfg = json.loads(path.read_text())
repo = "Qwen/Qwen3-4B"
sha = HfApi().model_info(repo, revision="main").sha
cfg.update(
    model_repo=repo,
    model_revision=sha,
    model_dir="models/Qwen3-4B-" + sha[:12],
    served_model="qwen3-4b-local",
)
path.write_text(json.dumps(cfg, ensure_ascii=False, indent=2) + "\n")
PY
python scripts/download_model.py
bash scripts/start.sh
bash scripts/test_qwen.sh
python scripts/test_agents.py
bash scripts/run_demo.sh

```

这里被替换的四项分别是原 `model_repo="Qwen/Qwen3-8B"`、旧的 `model_revision`、`model_dir="models/Qwen3-8B"`、`served_model="qwen3-8b-local"`。没有让你手填一个未经验证的 commit；脚本查询真实存在的快照再固定它。

9.4 换成别的模型家族时，Python 哪些地方可能要改？

`service.py` 的 `cmd` 列表控制 `--dtype bfloat16`、上下文、推理后端等；不同模型可能不支持当前设置。`client.py` 的 `extra_body` 里 `chat_template_kwargs.enable_thinking` 是 Qwen3 的对话模板约定，不能假定任何模型都支持。对没有这个选项、但仍支持 `guided_json` 的模型，可把原语句：

```
extra_body={"chat_template_kwargs": {"enable_thinking": False}, "guided_json": schema})
```

替换为：

```
extra_body={"guided_json": schema})
```

这个条件式示例只移除 Qwen 特有的模板参数，不保证新模型已被旧 vLLM 支持。也不建议在当前 Qwen3 模型上随意移除，因为默认思考内容可能影响纯 JSON 返回和 token 长度。若要启用 Qwen 思考模式，不能只把 `False` 改 `True` 就宣布完成：需正确处理 reasoning 与最终答案分离、预算和结构化输出兼容性。

模型的内置 chat template 位于 `tokenizer_config.json` 等模型文件，由服务自动使用。当前程序没有单独的 prompt template 引擎；`prompts/*.md` 只是角色内容。换家族时先阅读该模型官方卡、服务版本的支持列表，再决定配置和代码修改，不能由一个 `model_repo` 字符串解决全部兼容问题。

9.5 升级 vLLM 或 PyTorch，应该修改哪里？

`requirements.txt` 中的 `vllm==0.8.5` 是直接依赖声明；`requirements.lock` 中也固定了 `vllm` 及其大量间接依赖。`install.sh` 优先读取 `requirements.lock`，所以只改 `requirements.txt` 再运行 `install.sh`，仍会装锁文件中的旧版本。`service.py` 的命令行参数、`client.py` 的 `guided_json` 参数也可能随推理版本变化，需要一起核对。

没有一个可对任意未来版本照抄的“把 0.8.5 改成最新版”方案：新框架可能要求高于现有驱动的 CUDA。这里保留已验证版本。可在项目内新建测试环境，不覆盖 .venv：

```
cd /home/xu
source scripts/activate.sh
.runtime/tools/bin/uv venv --python 3.11.13 .venv-trial
```

然后为你已查证兼容的目标版本建立 requirements.trial.txt，使用 `uv pip install --python .venv-trial/bin/python -r requirements.trial.txt` 安装。具体版本必须根据当时官方兼容说明选定，本手册不猜测未来版本。不要把这一命令当成已完成升级。要让启动和实验脚本使用测试环境，还需把对应 .sh 中 "`$PROJECT_ROOT/.venv/bin/python`" 明确换为 "`$PROJECT_ROOT/.venv-trial/bin/python`"; activate.sh 中的 `.venv/bin/activate` 也需要配套调整。

确认试验环境通过 GPU、API 和端到端测试后，才导出该环境自己的 `pip freeze --all` 清单作为新的锁文件；不要从旧环境导出后误认为锁住新版本。若测试环境没有 pip，可用 `.runtime/tools/bin/uv pip freeze --python .venv-trial/bin/python` 导出。全程不得通过升级系统驱动解决兼容性问题。

9.6 模型切换失败如何回退？

```
cd /home/xu
source scripts/activate.sh
bash scripts/stop.sh
cp docs/settings.before-4b.json config/settings.json
bash scripts/start.sh
bash scripts/test_qwen.sh
```

上例对应 9.3 的备份文件；如果你做的是 9.2，应使用 settings.before-model-update.json。旧权重目录仍在，因此恢复配置后可以重新加载原模型。不要删除旧权重再测试新模型，否则回退可能还要重新下载。

10. 遇到问题，按文件定位

现象	优先看哪里	含义与下一步
No module named ...	which python ; .venv ; requirements.lock	可能没有使用项目解释器，先 source activate.sh
attempted relative import ...	run_demo.sh、run.py 启动方式	使用 -m analog_agents.run，不要直接运行包内文件路径
JSONDecodeError	settings.json 或模型 message.content	配置语法错误或回答不是完整 JSON；区分发生在哪个 json.loads
ValidationError	agents.py 的 SCHEMAS、原始参数	缺字段、类型不符或超出范围；不是 GPU 坏了
Connection error	logs/qwen.log、端口配置	模型未启动、已崩溃、端口不一致，或工具沙盒限制
GPU is busy	service.py 查询结果、nvidia-smi	当前卡有其他任务；等待或选择已确认空闲的卡
Model absent	model_dir/config.json	尚未下载到配置指定目录，或路径写错
Incomplete model output: length	client.py、max_tokens	模型输出到达长度上限；可能需减少输入或增加合理预算
Context length exceeded	context_length 与请求历史大小	过多历史或长提示词；减小 payload 或调整容量
summary completed 但 gain_db=false	history.json、Mock 公式	流程结束而目标未达，不是成功标志互相矛盾
改 prompt 没变化	prompts 的文件名、当前进程	已创建的 Agent 使用内存文本；重新运行实验
改 port 后无法连接	service.py 与 client.py 的配置读取时机	旧服务仍监听旧端口；停止再启动

检查文件可使用 cat 查看较短 JSON，tail 查看日志末尾。不要把未知网络教程中的 sudo、全局 pip、全局 CUDA 升级或 killall 命令直接复制到共享服务器。这里只需围绕项目脚本和已知日志定位。

```
cd /home/xu
tail -n 80 logs/qwen.log
cat logs/verification.json
cat outputs/20261002-131535-070986/summary.json
```

11. 如何阅读后面的逐行附录

每个文件先说明它的角色和被谁调用，然后列出原始行号、原始代码与中文解释。缩进是 Python 语法的一部分；括号未闭合时，同一条语句可以跨越多行，不能把某个续行单独执行。空行没有执行动作，但用于分组；# 后的内容通常是注释；三引号在某些位置是说明字符串，在 netlist() 中则是需要返回的多行电路文本。

import、def、class 之后的执行时机不同：模块被 import 时顶层 import/赋值会执行，def 的函数正文等到被调用才执行。if __name__ == "__main__" 避免模块被别人导入时自动运行整个程序。for 遍历一组值；if 分支选择下一步；break 退出循环；try/except 捕获并记录错误；raise 让错误继续中断当前任务。

本附录还逐行解释自动生成的激活脚本、安装/下载辅助流程和离线测试。这些代码不是每次实验都全部执行。例如 install.sh、download_model.py 只在安装或主动更新时用；pytest 测试不调用 Qwen；build_guide.py 只生成文档，不属于被解释的 Agent workflow。

已部署的 API/Agent/GPU 测试证据仍在 logs/verification.json；本次只更新文档及文档生成工具，没有重新运行 GPU 实验。修改示例的静态/局部验证会单独记录，不能替代你实际改代码后的集成测试。

附录 A：当前源文件逐行解释

每行代码均来自本次文件快照。以下解释覆盖 26 个文件共 586 行；空行也保留行号。长行的 PDF 自动折行不改变源文件行号。

A01. scripts/activate.sh

用户用 source 运行，修改当前终端的环境。它本身不启动模型、不运行实验。

```
L001 #!/usr/bin/env bash
```

声明 Bash 脚本；当前终端 source 时作为注释处理。

```
L002 source "$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)/env.sh"
```

从本脚本所在目录找 env.sh 并在当前 shell 中执行，让 PROJECT_ROOT 与缓存变量生效；子命令里的 cd 不把终端留在 scripts 目录。

```
L003 source "$PROJECT_ROOT/.venv/bin/activate"
```

在当前 shell 执行项目虚拟环境的自动激活脚本，将 python 等命令优先指向 .venv。完整自动脚本也在后面附录中解释。

A02. scripts/env.sh

每个日常 shell 入口都会 source 本文件，统一环境变量和缓存路径。export 让随后启动的 Python 子进程也继承这些设置。

L001 `#!/usr/bin/env bash`

shebang 声明用 Bash 解释脚本；被 source 时由当前 shell 解释，此行作为注释不另起进程。

L002 `export PROJECT_ROOT="$(cd -- "${dirname -- "${BASH_SOURCE[0]}"}" && pwd -P)"`

BASH_SOURCE[0] 取脚本路径，dirname 取 scripts 目录，/.. 上到根目录；子命令在自己的上下文中 cd，pwd -P 给出真实目录。export PROJECT_ROOT 把结果传给后续程序，不改变用户 HOME。

L003 `export TMPDIR="$PROJECT_ROOT/.runtime/tmp"`

将标准临时目录变量 TMPDIR 指向项目 .runtime/tmp，Python tempfile 等组件通常据此选择临时目录。

L004 `export TMP="$TMPDIR" TEMP="$TMPDIR"`

同时设置 TMP 和 TEMP，兼容使用其他临时目录变量名的软件；值都与 TMPDIR 相同。

L005 `export XDG_CACHE_HOME="$PROJECT_ROOT/.runtime/cache"`

把遵守 XDG 约定的软件缓存放在项目 .runtime/cache。

L006 `export XDG_CONFIG_HOME="$PROJECT_ROOT/.runtime/config"`

把遵守 XDG 约定的软件配置放在项目 .runtime/config。

L007 `export XDG_DATA_HOME="$PROJECT_ROOT/.runtime/data"`

把遵守 XDG 约定的持久数据放在项目 .runtime/data。

L008 `export XDG_STATE_HOME="$PROJECT_ROOT/.runtime/state"`

把遵守 XDG 约定的运行状态放在项目 .runtime/state。

L009 `export UV_CACHE_DIR="$XDG_CACHE_HOME/uv"`

为 uv 指定项目内缓存目录，供下载安装时使用。

L010 `export UV_PYTHON_INSTALL_DIR="$PROJECT_ROOT/.runtime/python"`

指定 uv 安装独立 Python 解释器的位置 .runtime/python。

L011 `export UV_PYTHON_BIN_DIR="$PROJECT_ROOT/.runtime/bin"`

指定 uv Python 可执行文件链接的存放位置 .runtime/bin；不要求添加到全局 shell 配置。

L012 `export PIP_CACHE_DIR="$XDG_CACHE_HOME/pip"`

指定 pip 缓存路径；某些安装命令还使用 --no-cache-dir 主动关闭该缓存。

L013 `export HF_HOME="$XDG_CACHE_HOME/huggingface"`

指定 Hugging Face 的缓存/本地配置根目录，避免使用默认的其他位置；模型权重另有显式 local_dir。

L014 `export HF_HUB_DISABLE_IMPLICIT_TOKEN=1 HF_HUB_DISABLE_TELEMETRY=1`

禁用自动附带已有 token 和 Hugging Face 遥测；下载公开模型不需要你提供云端 API 密钥。

L015 `export TORCH_HOME="$XDG_CACHE_HOME/torch"`

指定 PyTorch 自身的缓存目录。

L016 `export TORCH_EXTENSIONS_DIR="$XDG_CACHE_HOME/torch_extensions"`

指定 PyTorch 扩展编译产物的目录，即使某项依赖需要生成扩展也留在项目内。

L017 `export TRITON_CACHE_DIR="$XDG_CACHE_HOME/triton"`

指定 Triton 内核编译缓存目录，用于 GPU 运算相关依赖。

L018 `export CUDA_CACHE_PATH="$XDG_CACHE_HOME/cuda"`

指定 NVIDIA CUDA 编译缓存路径。

L019 `export VLLM_CACHE_ROOT="$XDG_CACHE_HOME/vllm"`

指定 vLLM 缓存根目录。

L020 `export VLLM_CONFIG_ROOT="$PROJECT_ROOT/.runtime/config/vllm"`

指定 vLLM 配置根目录，和模型权重目录不同。

L021 export NUMBA_CACHE_DIR="\$XDG_CACHE_HOME/numba"

指定 Numba 的编译缓存目录。

L022 export MPLCONFIGDIR="\$XDG_CACHE_HOME/matplotlib"

指定 Matplotlib 配置/缓存目录；当前日常 Agent 流程未绘图，但预先约束可能使用它的库。

L023 export PYTHONNOUSERSITE=1 PYTHONDONTWRITEBYTECODE=1

PYTHONNOUSERSITE 避免从用户默认 site-packages 混入包；PYTHONDONTWRITEBYTECODE 阻止生成 .pyc 字节码缓存，但不禁止所有文件写入。

L024 export VLLM_NO_USAGE_STATS=1 DO_NOT_TRACK=1 ANONYMIZED_TELEMETRY=False

关闭 vLLM 使用统计，并给识别这些变量的组件声明不追踪；并非操作系统级的网络阻断器。

L025 export OMP_NUM_THREADS=4 TOKENIZERS_PARALLELISM=false

限制 OpenMP CPU 线程为 4，并禁用分词器并行，减少共享 CPU 资源争用；不代表 vLLM 所有内部线程都精确只有 4 个。

L026 export NCCL_SHM_DISABLE=1 VLLM_HOST_IP=127.0.0.1

禁用 NCCL 的共享内存传输路径，设置 vLLM 主机地址为本机回环。GPU 运算仍可运行；不是禁用 CUDA。

L027 export PYTHONPATH="\$PROJECT_ROOT"

让 Python 的模块搜索路径包括项目根目录，这样直接运行 scripts/ 下的文件也能 import analog_agents。

L028 mkdir -p "\$TMPDIR" "\$XDG_CACHE_HOME" "\$XDG_CONFIG_HOME" "\$XDG_DATA_HOME" "\$XDG_STATE_HOME"

创建最基本的临时、缓存、配置、数据、状态目录；mkdir -p 在目录已存在时不报错，也不删除旧内容。许多更深的缓存子目录由对应库按需创建。

A03. scripts/start.sh

日常 Bash 入口。使用绝对的虚拟环境 Python 路径，即使没有手动激活也不会意外选中系统 Python；仍需先启动模型才能调用 API。

L001 #!/usr/bin/env bash

shebang 声明这是 Bash 文件；日常用 bash 文件名 显式解释。

L002 set -euo pipefail

set -e 让通常的命令失败中断脚本，-u 拒绝未定义变量，pipefail 使管道中间失败也可传播；Bash 某些条件语境有例外，不能看作完整异常处理。

L003 source "\$(dirname -- "\$0")/env.sh"

根据脚本文件位置 source 同目录 env.sh，统一 PROJECT_ROOT 和缓存/临时环境。

L004 cd "\$PROJECT_ROOT"

把当前子进程工作目录切到 PROJECT_ROOT，使后续 scripts/... 相对路径有确定含义。

L005 exec "\$PROJECT_ROOT/.venv/bin/python" scripts/service.py start

用 exec 将当前 shell 进程替换成项目 Python，执行 scripts/service.py，传入 start；之后由服务管理代码创建后台 vLLM。

A04. scripts/service.py

模型服务的启动和停止管理器。它会读配置、GPU 状态和服务状态，启动第三方 vLLM 入口；本文件自己不实现神经网络。

```
L001 """Single-GPU, loopback-only service. Stop only the process group we created."""
```

模块说明：仅单 GPU、仅本机接口，只停止自己创建的进程组。三引号说明不执行管理操作。

```
L002 import fcntl
```

导入 Linux 文件锁接口 fcntl，防止两个管理命令同时操作状态。

```
L003 import json
```

导入 json，用于保存和读取服务 PID/启动命令等数据。

```
L004 import os
```

导入 os，用于复制环境变量和向进程组发信号。

```
L005 import signal
```

导入 signal，使用有名称的 SIGTERM 常量，要求进程正常终止。

```
L006 import socket
```

导入 socket，用于启动前临时检查监听端口能否绑定。

```
L007 import subprocess
```

导入 subprocess，用于运行 nvidia-smi 和创建 vLLM 后台子进程。

```
L008 import sys
```

导入 sys，读取当前 Python 可执行路径 sys.executable 和用户命令参数 sys.argv。

```
L009 import time
```

导入 time，为启动/停止的轮询之间加入短暂等待。

```
L010 import urllib.request
```

导入标准库 HTTP 请求工具，用来访问本地 /health 健康检查接口。

```
L011 from analog_agents.config import load_config, project_path, ROOT
```

从项目配置模块导入配置读取、路径检查和根目录常量。

```
L012 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L013 STATE = project_path("logs/service.json")
```

计算服务状态文件路径 logs/service.json，保存为 STATE。此刻没有读取或创建该文件。

```
L014 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L015 def birth(pid):
```

定义 birth(pid)，查询一个 Linux 进程的启动时刻标记，以识别 PID 是否被重用。

```
L016     try:
```

尝试读取进程信息；进程可能已经退出，所以必须处理文件不存在。

```
L017         return open(f"/proc/{pid}/stat").read().split(" ", 1)[1].split()[19]
```

读取 /proc/PID/stat，先跳过括号包围的进程名，再按空格拆分，从后半部分第 20 项取原 stat 的 starttime 字段。这个值与 PID 组合用来区分不同生命周期的进程。

```
L018     except FileNotFoundError:
```

如果 /proc 文件不存在，说明查询时进程可能已经消失，进入这一分支。

```
L019         return None
```

返回 None 表示未取到出生标记，不是创建新进程。

```
L020 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L021 def owned(state):
```

定义 owned(state)，判断状态文件记录的进程与当前同 PID 的进程是否仍是同一个。

```
L022     return birth(state["pid"]) == state["birth"]
```

比较当前查询的 birth 与保存值；相同则返回 True。该方法依赖项目状态文件完整可信，不是通用操作系统所有权认证。

```
L023     [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L024 def stop():
```

定义 stop 操作，下面正文等 service.py stop 分派到它时才执行。

```
L025     if not STATE.exists():
```

先判断服务状态文件是否不存在。

```
L026         print("No managed service.")
```

无状态文件则打印“没有受管理的服务”，不会全局搜索并杀 Python。

```
L027         return
```

提前返回，结束 stop。

```
L028     state = json.loads(STATE.read_text())
```

有状态文件则读取 JSON，得到 pid、birth 等信息。

```
L029     if owned(state):
```

仅当进程出生标记吻合时，才执行下一步发信号操作。

```
L030         os.killpg(state["pid"], signal.SIGTERM)
```

向 PID 对应的进程组发送 SIGTERM。启动时 start_new_session=True 使该组属于本项目启动的服务；并不是按进程名匹配全部用户进程。

```
L031         for _ in range(60):
```

最多检查 60 次，_ 表示不使用这个循环序号。

```
L032             if not owned(state):
```

查询当前 PID 是否已经不再对应记录进程。

```
L033                 break
```

进程已不在则退出等待循环，继续清理状态文件。

```
L034             # A reaped-by-parent zombie cannot hold GPU resources.
```

注释意图是把僵尸进程视为已退出。严格说 zombie 是已结束但尚待父进程回收的状态，注释中 reaped-by-parent 的措辞不严谨；此处不再继续等它。

```
L035             stat = open(f"/proc/{state['pid']}/stat").read().split(" ", 1)[1]
```

读取进程状态文本，去掉进程名部分，为检查 Z 状态准备。

```
L036             if stat.startswith("Z"):
```

若后半部分以 Z 开头，说明进程是 zombie，已不在执行模型计算。

```
L037                 break
```

离开等待循环。GPU 释放和 nvidia-smi 更新仍可能有短暂延迟，需要另行查看。

```
L038             time.sleep(0.5)
```

若进程仍活动，等 0.5 秒再检查。

```
L039     else:
```

这是 Python for...else：只有循环用完 60 次而没有 break 时才执行，不是上面某个 if 的 else。

```
L040         raise RuntimeError("Graceful stop timed out; inspect logs before any further action")
```

停止超时则抛错，让人查看日志，不会直接升级为强制 SIGKILL。

```
L041     STATE.unlink()
```

删除本项目服务状态文件；不删除模型、日志或实验输出。若记录 PID 不再属于原进程，也只清理这份旧状态。

```
L042     print("Managed Qwen service stopped.")
```

打印受管理服务已停止。并非系统全部 GPU 都一定空闲的证明。

L043 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L044 `def start():`

定义 start 操作，用于启动模型服务。

L045 `c = load_config()`

从 config/settings.json 读取集中配置，接下来使用 c 这个较短变量名。

L046 `if STATE.exists() and owned(json.loads(STATE.read_text())):`

如果状态文件存在并且仍对应同一进程，认为服务已启动。

L047 `raise RuntimeError("Managed service already running; use stop.sh first")`

抛出已有服务错误，避免同项目重复分配模型显存。

L048 `with socket.socket() as sock:`

创建临时 socket，with 结束后自动关闭。

L049 `sock.bind(("127.0.0.1", c["port"]))`

尝试绑定本机指定端口；端口已用时会抛 OSError。socket 很快关闭，所以这不是直到真正启动之间的原子端口预约。

L050 `lines = subprocess.check_output(["nvidia-smi", f"--id={c['gpu']}"],`

运行 nvidia-smi 命令并选择配置中的卡号；使用列表传命令参数，不通过 shell 执行任意拼接代码。

L051 `--query-gpu=uuid,memory.used,memory.total,utilization.gpu", "`
`--format=csv,noheader,nounits"], text=True).strip()`

继续指定查询 UUID、显存已用、总量和利用率，要求无表头无单位 CSV；text=True 得到字符串，strip 去掉首尾空白。

L052 `uuid, used, total, util = [s.strip() for s in lines.split(",")]`

按逗号拆分返回值并逐项去空白，分别赋给 uuid/used/total/util。当前 total 被读取但没有用于额外显存充足性判断。

L053 `if int(used) > 1024 or int(util) > 5:`

将已用显存和利用率转成整数，超过 1024 MiB 或 5% 就认为忙碌。

L054 `raise RuntimeError(f"GPU {c['gpu']} is busy: {lines}. Choose an idle GPU; no process will be killed.")`

忙碌则抛错并报告观测值；不尝试终止其他作业，也不自动选择另一张卡。

L055 `model = project_path(c["model_dir"])`

把模型目录配置解析成经过边界检查的项目绝对 Path。

L056 `if not (model / "config.json").exists():`

查看模型目录里的 config.json 是否存在，这只是最小存在性检查。

L057 `raise RuntimeError("Model absent. Run scripts/download_model.py first.")`

缺少配置就报错，提醒先下载模型；不在启动过程偷偷联网下载。

L058 `env = os.environ.copy()`

复制当前环境变量字典，后面只修改子进程使用的 env，不修改系统环境配置。

L059 `# vLLM 0.8.5's NVML mapper requires a numeric CUDA_VISIBLE_DEVICES value.`

注释说明 vLLM 0.8.5 的 NVML 映射需要数字 CUDA_VISIBLE_DEVICES；先前使用 UUID 曾失败，因此这里使用编号。

L060 `env.update(CUDA_VISIBLE_DEVICES=str(c["gpu"]), CUDA_DEVICE_ORDER="PCI_BUS_ID",`
`VLLM_USE_V1="0",`

对子进程限制可见 GPU 为配置的一张卡，采用 PCI_BUS_ID 排序；VLLM_USE_V1=0 指定该版本的 V0 引擎路径。

L061 `HF_HUB_OFFLINE="1", TRANSFORMERS_OFFLINE="1", VLLM_WORKER_MULTIPROC_METHOD="spawn")`

设置 Hugging Face/Transformers 离线，避免服务启动时访问远程；设置工作进程启动方法 spawn。右括号结束 env.update。

L062 `cmd = [sys.executable, "-m", "vllm.entrypoints.openai.api_server", "--model", str(model),`
建立要运行的命令列表；sys.executable 是当前 .venv Python，-m 后面给第三方 vLLM 模块入口，--model 指向本地权重目录。

L063 `"--served-model-name", c["served_model"], "--host", "127.0.0.1", "--port", str(c["port"]],`
设置 API 对外模型别名、监听地址仅 127.0.0.1，以及端口。

L064 "--dtype", "bfloat16", "--max-model-len", str(c["context_length"]),
权重/计算类型采用 bfloat16；最大输入加输出上下文使用配置值。BF16 是 16 位浮点表示，不是模型名称。

L065 "--gpu-memory-utilization", str(c["gpu_memory_utilization"]), "--tensor-parallel-size", "1",
设置显存预算比例和张量并行大小 1，只用单卡而不是分到多卡。

L066 "--max-num-seqs", "2", "--enforce-eager", "--disable-log-requests", "--disable-frontend-multiprocessing",
同时处理的最大序列数为 2；enforce-eager 使用直接执行模式而非 CUDA 图捕获；关闭请求内容日志，关闭前端多进程。
服务仍会输出运行和访问日志。

L067 "--guided-decoding-backend", "xgrammar"]
结构化生成后端设置 xgrammar；该 vLLM 版本对某些复杂 Schema 可能内部回退，最终客户端仍会 jsonschema 校验。

L068 with project_path("logs/qwen.log").open("a") as log:
以追加模式打开 logs/qwen.log，保留旧启动记录，模型的输出写到这里。

L069 proc = subprocess.Popen(cmd, env=env, cwd=ROOT, stdout=log, stderr=subprocess.STDOUT, start_new_session=True)
真正创建后台 vLLM 进程：传递 env、工作目录 ROOT，把标准错误也并入同一日志；start_new_session 建立新会话，使启动管理器退出后服务继续运行。

L070 STATE.write_text(json.dumps({"pid": proc.pid, "birth": birth(proc.pid), "gpu_uuid": uuid, "port": c["port"], "command": cmd}, indent=2))
记录 PID、出生标记、GPU UUID、端口和完整启动参数到状态文件；indent=2 便于人工阅读。

L071 print(f"Starting PID {proc.pid} on GPU {c['gpu']} ({uuid}); see logs/qwen.log", flush=True)
立即打印正在启动及 PID，提示用户查看日志；这还不是 Ready。

L072 for _ in range(300):
最多检查健康状态 300 次；每次还有请求超时与 sleep，因此不是严格 600 秒的硬上限。

L073 if proc.poll() is not None:
poll 查询后台进程是否已退出。返回 None 表示还在运行，非 None 是退出码。

L074 STATE.unlink(missing_ok=True)
若服务已退出，移除失效状态文件；missing_ok=True 表示文件已不存在也不报错。

L075 raise RuntimeError(f"Server exited with {proc.returncode}; see logs/qwen.log")
抛出服务启动失败，并提示查看 qwen.log，保留具体退出码。

L076 try:
尝试访问健康端点，加载期间连不上是可以暂时接受的情况。

L077 with urllib.request.urlopen(f"http://127.0.0.1:{c['port']}/health", timeout=2) as response:
发 GET 请求到本机 /health，单次连接/读取设置 2 秒超时；with 自动关闭响应对象。

L078 if response.status == 200:
HTTP 状态码 200 表示服务健康检查通过。

L079 print(f"Ready: http://127.0.0.1:{c['port']}/v1")
打印 Ready 和 API 基础地址，告诉使用者可以开始发送请求。

L080 return
结束 start 函数，随后启动管理脚本结束；后台 vLLM 不因此停止。

L081 except (OSError, TimeoutError):
捕获临时网络/操作系统错误和超时，常见于服务仍在加载模型。

L082 pass
pass 表示本次忽略该临时失败，不做其他动作，继续后续等待。

L083 time.sleep(2)
每轮等 2 秒，避免用紧密死循环不断请求服务。

L084 stop()

检查次数耗尽仍未成功时，调用本文件 stop，尝试清理自己启动的服务。

L085 raise RuntimeError("Service startup timed out")

清理后抛出启动超时错误，不能把这次启动标为成功。

L086 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L087 if __name__ == "__main__":

只有脚本作为主程序执行时，才进行命令分派；import 本文件不会自动启动 GPU。

L088 with project_path("logs/service.lock").open("w") as lock:

打开项目的 service.lock 文件作为互斥锁载体；with 结束时关闭文件并释放锁。

L089 fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)

申请排他且非阻塞文件锁：若另一个管理操作持有锁，立即报错而不是一直等待。该锁不等于系统 GPU 排队机制。

L090 {"start": start, "stop": stop}[sys.argv[1]]()

构建 start/stop 对应函数字典，按 sys.argv[1] 选出函数并调用；argv[0] 是脚本名。未提供参数或拼错参数会报错，没有完整命令行帮助解析器。

A05. config/settings.json

用户集中配置。文件是 JSON 数据而非 Python。run.py 和 service.py 分别在各自启动时读取，因此更改服务相关字段需要重启后台。

```
L001 {  
左花括号开始最外层 JSON 对象；字段名要双引号，文件中不能添加 # 注释。  
  
L002   "model_repo": "Qwen/Qwen3-8B",  
远程模型仓库名称。下载脚本使用它，运行中的服务主要使用本地 model_dir，不因改名字自动换权重。  
  
L003   "model_revision": "b968826d9c46dd6066d109eabc6255188de91218",  
固定的 Hugging Face 提交号，锁定下载快照。它不是 Python/vLLM 版本号，也不是本地文件路径。  
  
L004   "model_dir": "models/Qwen3-8B",  
模型权重保存在项目内的相对目录；启动服务将它转成 /home/xu/models/Qwen3-8B。  
  
L005   "served_model": "qwen3-8b-local",  
API 提供给客户端的模型别名。service.py 与 client.py 都用这个字段保持一致。  
  
L006   "gpu": 0,  
选择 nvidia-smi 中编号 0 的 GPU；这是一个整数，不是四张卡同时使用的数量。  
  
L007   "port": 8000,  
本地服务端口号。服务和客户端都由此决定地址，更改后要重启旧服务。  
  
L008   "context_length": 8192,  
最大上下文 token 长度 8192，包括输入和生成输出，并非字符数量。  
  
L009   "gpu_memory_utilization": 0.5,  
vLLM 显存预算比例 0.5；实际显存有开销，且这不是 GPU 运算负载上限。  
  
L010   "max_tokens": 1200,  
单次回答最多生成 1200 token，过小可能让 JSON 被截断；不是整个实验的 token 总预算。  
  
L011   "temperature": 0.2,  
采样温度 0.2，用于调节随机性，不是显卡温度。  
  
L012   "timeout_seconds": 180,  
SDK 网络超时设置 180 秒；多次请求与重试使整个 workflow 可能远超过这个时长。  
  
L013   "max_iterations": 2,  
最多允许两次参数更新，主循环因包含初始评估最多计算三轮。  
  
L014   "simulation_backend": "mock",  
仿真后端名为 mock。当前 Python 代码只接受它，改为其他文字不会自动获得新工具。  
  
L015   "specification": {"circuit": "two_stage_cmos_opamp", "vdd_v": 1.8, "load_pf": 5.0, "  
gain_db_min": 65, "ugb_mhz_min": 10, "phase_margin_deg_min": 60, "power_mw_max": 1.0}  
设计规格：circuit 是任务标签；vdd_v=1.8 伏，load_pf=5 皮法；增益至少 65 dB、单位增益带宽至少 10 MHz、相位裕度至少 60 度、功耗最多 1 mW。当前结果是合成指标。  
  
L016 }
```

右花括号结束 JSON 对象。最后字段后不应留下多余逗号。

A06. scripts/test_qwen.sh

日常 Bash 入口。使用绝对的虚拟环境 Python 路径，即使没有手动激活也不会意外选中系统 Python；仍需先启动模型才能调用 API。

```
L001 #!/usr/bin/env bash  
shebang 声明这是 Bash 文件；日常用 bash 文件名 显式解释。
```

L002 `set -euo pipefail`

`set -e` 让通常的命令失败中断脚本，`-u` 拒绝未定义变量，`pipefail` 使管道中间失败也可传播；Bash 某些条件语境有例外，不能看作完整异常处理。

L003 `source "${dirname -- "$0"}/env.sh"`

根据脚本文件位置 `source` 同目录 `env.sh`，统一 `PROJECT_ROOT` 和缓存/临时环境。

L004 `cd "$PROJECT_ROOT"`

把当前子进程工作目录切到 `PROJECT_ROOT`，使后续 `scripts/...` 相对路径有确定含义。

L005 `exec "$PROJECT_ROOT/.venv/bin/python" scripts/test_qwen.py`

用 `exec` 执行项目 Python 的 `scripts/test_qwen.py`，运行一次真实本地 API 测试。

A07. scripts/test_qwen.py

实际调用本地服务的算术冒烟测试。它验证通信、生成、JSON 结构和一个简单答案，不评价电路能力。

```
L001 import json
```

导入 json，准备写测试结果文件。

```
L002 from analog_agents.config import load_config, project_path
```

导入配置读取与日志路径检查。

```
L003 from analog_agents.client import LocalClient
```

导入统一 API 客户端，保证测试走同一通信代码。

```
L004 from analog_agents.agents import obj
```

导入 obj Schema 辅助函数，构造简单输出结构。

```
L005 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L006 def main():
```

定义测试主函数，避免被 pytest 或其他文件导入时立刻连接服务。

```
L007     schema = obj({"answer": {"type": "integer"}})
```

要求响应是只有 answer 必填整数的对象；additionalProperties 等规则由 obj 自动添加。

```
L008     answer = LocalClient(load_config()).ask("Compute the sum, return JSON.", {"question": "19+23=?"}, schema)
```

读取配置、创建本地客户端并发送 19+23 问题；返回值经 ask 内的 JSON 解析与校验后才赋给 answer。

```
L009     assert answer["answer"] == 42, answer
```

检查 answer 字段是否为 42，否则抛 AssertionError 并显示实际回答。

```
L010     project_path("logs/api_test.json").write_text(json.dumps({"status": "passed", "response": answer}))
```

只有上面的断言通过才写 status=passed 和实际响应到 logs/api_test.json。

```
L011     print("Local Qwen API: PASS", answer)
```

在终端打印 PASS 和回答，便于用户快速确认。

```
L012 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L013 if __name__ == "__main__":
```

只有作为主程序执行才运行下面的 main，import 时不触发网络请求。

```
L014     main()
```

调用 main 开始测试。

A08. scripts/run_demo.sh

日常 Bash 入口。使用绝对的虚拟环境 Python 路径，即使没有手动激活也不会意外选中系统 Python；仍需先启动模型才能调用 API。

```
L001 #!/usr/bin/env bash
```

shebang 声明这是 Bash 文件；日常用 bash 文件名 显式解释。

```
L002 set -euo pipefail
```

set -e 让通常的命令失败中断脚本，-u 拒绝未定义变量，pipefail 使管道中间失败也可传播；Bash 某些条件语境有例外，不能看作完整异常处理。

```
L003 source "${dirname -- "$0"}/env.sh"
```

根据脚本文件位置 source 同目录 env.sh，统一 PROJECT_ROOT 和缓存/临时环境。

```
L004 cd "$PROJECT_ROOT"
```

把当前子进程工作目录切到 PROJECT_ROOT，使后续 scripts/... 相对路径有确定含义。

```
L005 exec "$PROJECT_ROOT/.venv/bin/python" -m analog_agents.run "$@"
```

用 exec 执行项目 Python，-m analog_agents.run 表示作为包模块启动主流程。"\$@" 原样转发额外命令行参数；当前 run.py 没有参数解析，因此附加 --foo 等选项不会自动改变配置。

A09. analog_agents/__init__.py

包初始化文件。import analog_agents 或执行 -m analog_agents.run 时会经过它；这里只有说明，不运行实验。

```
L001 """Local Qwen agents for analog circuit workflow experiments."""
```

三引号包说明：本包是使用本地 Qwen 进行模拟电路 workflow 实验的 Agent 代码。没有函数调用、网络请求或 GPU 操作；Python 把它保存为模块文档字符串。

A10. analog_agents/config.py

统一读取项目配置并限制项目文件路径。启动服务、API 客户端测试和正式实验都会用它。

```
L001 import json
```

导入 Python 标准库 json，后面将 settings.json 文本转换为 Python 字典；不是安装新库。

```
L002 from pathlib import Path
```

从标准库 pathlib 取出 Path 类，用它组合、解析和读取文件路径。

```
L003 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L004 ROOT = Path(__file__).resolve().parents[1]
```

__file__ 是当前 config.py 的文件名；resolve 得到真实绝对路径；parents[0] 是 analog_agents 文件夹，parents[1] 是 /home/xu，因此 ROOT 是项目根目录。

```
L005 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L006 def project_path(relative):
```

定义 project_path 函数，输入 relative 是希望访问的项目相对路径，也能接收绝对路径；函数定义此刻不访问文件。

```
L007     path = (ROOT / relative).resolve()
```

Path 的 / 运算连接 ROOT 和 relative，再 resolve 消除 .. 并解析符号链接。若输入是绝对路径，Path 会采用该绝对路径，所以下行仍必须检查边界。

```
L008     if not path.is_relative_to(ROOT):
```

判断解析后的真实路径是否不在 ROOT 内。not 表示取反，冒号后的缩进部分仅在越界时执行。

```
L009         raise ValueError(f"Path escapes project: {relative}")
```

越界就抛 ValueError，中断本次操作；f 字符串将请求路径填入报错文字。这防止配置路径逃出项目，不是限制所有第三方系统调用的操作系统沙盒。

```
L010     return path
```

路径合法时 return 返回 Path 对象。这里只算路径，并没有打开或创建文件。

```
L011 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L012 def load_config():
```

定义 load_config 函数，负责读取并检查集中配置。

```
L013     cfg = json.loads(project_path("config/settings.json").read_text())
```

先 project_path 定位 config/settings.json，再 read_text 读文本，最后 json.loads 解析成字典 cfg。文件缺失或 JSON 错误会抛异常。

```
L014     project_path(cfg["model_dir"])
```

对 cfg 中 model_dir 再做项目边界检查；没有检查该目录的所有权重是否已下载。

```
L015     assert isinstance(cfg["gpu"], int) and cfg["gpu"] >= 0
```

assert 要求 gpu 是非负整数。and 要求两个条件同时为真；失败抛 AssertionError。assert 在 Python -O 优化模式会被禁用，本项目启动命令没有使用 -O。

```
L016     assert 1024 <= cfg["port"] <= 65535
```

限制端口在 1024—65535 之间，避免使用低编号特权端口。此行不检查端口是否已经被别的程序占用。

```
L017     assert 0.1 <= cfg["gpu_memory_utilization"] <= 0.6
```

限制 vLLM 显存预算比例在 0.1—0.6 之间。它不是限制 GPU 计算利用率。

```
L018     assert 1024 <= cfg["context_length"] <= 32768
```

限制上下文长度在 1024—32768 token 之间，这是项目允许范围，不是动态计算当前模型可承受的显存。

```
L019     assert 1 <= cfg["max_iterations"] <= 10
```

限制最多参数更新次数在 1—10 之间；主循环会额外加一次初始评估。

```
L020         assert cfg["simulation_backend"] == "mock", "Only explicitly labeled mock backend is implemented"
```

只允许 `simulation_backend` 为 `mock`，否则报出英文说明。目前改成 `ngspice` 不会自动接入真实仿真器。

```
L021         return cfg
```

把经过检查的字典 `cfg` 返回给调用者；后续代码通过 `cfg[字段名]` 使用设置。

A11. analog_agents/run.py

正式实验的 Python 入口，由 scripts/run_demo.sh 用 -m analog_agents.run 启动。负责顺序、数据传递、迭代、文件保存和错误报告。

```
L001 import json
```

导入 json，以便将配置、角色回答和历史记录写成 JSON 文本。

```
L002 from datetime import datetime
```

导入 datetime 类，用于取得当前时间并制作实验目录名。

```
L003 from zoneinfo import ZoneInfo
```

导入 ZoneInfo，让时间戳明确采用 Asia/Tokyo 时区，而不是任意机器的默认时区。

```
L004 from .config import load_config, project_path
```

导入配置读取与项目路径检查，所有主要输入/输出路径从这层定位。

```
L005 from .client import LocalClient
```

导入 LocalClient 类，后面为角色建立 API 客户端。

```
L006 from .agents import Agent
```

导入统一 Agent 类，后面用不同角色名实例化四次。

```
L007 from .simulation import MockSimulation
```

导入 MockSimulation 工具类；这明确决定当前执行的是 Mock，不是真实 SPICE。

```
L008 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L009 def main():
```

定义 main 主函数。文件底部会在直接执行模块时调用它。

```
L010     cfg = load_config()
```

实际调用 load_config，读取并检查 config/settings.json，结果保存在 cfg。

```
L011     stamp = datetime.now(ZoneInfo("Asia/Tokyo")).strftime("%Y%m%d-%H%M%S-%f")
```

获取日本时区当前时间并格式化成 年月日-时分秒-微秒 的字符串，减少多次实验目录重名。

```
L012     out = project_path(f"outputs/{stamp}")
```

拼接 outputs/时间戳 并做项目边界检查，得到输出目录 Path 对象 out。

```
L013     out.mkdir(parents=True)
```

真正创建实验目录；parents=True 允许同时创建缺失的上级目录。没有 exist_ok=True，若精确同名目录已存在会报错。

```
L014     def save(name, value):
```

在 main 内定义辅助函数 save，两个输入是文件名和待保存的数据；它能使用外层 out。

```
L015         (out / name).write_text(json.dumps(value, ensure_ascii=False, indent=2))
```

把 value 转成带两空格缩进、保留中文的 JSON 文本，并写到 out/name。write_text 覆盖已有同名文件，不是追加。

```
L016     save("config.json", cfg)
```

保存本次配置快照为实验目录的 config.json，方便以后复现设置。

```
L017     agents = {name: Agent(name, LocalClient(cfg)) for name in ("architecture", "sizing", "simulation", "optimization")}
```

字典推导式：依次为四个名字创建 LocalClient(cfg) 和 Agent，再形成“名字 -> Agent 对象”的字典。客户端有四个，后端模型服务仍是同一个。

```
L018     context = {"specification": cfg["specification"]}
```

创建初始上下文字典，只含设计 specification。以后还会把架构、尺寸回答加进去。

```
L019     history = []
```

创建空列表 history，用来记录各轮已经计算的结果。

```
L020     try:
```

开始异常保护区。这里之后、except 之前的错误会尝试写 failure.json；前面读取配置或读取提示词出错不在这个保护区。

```

L021         for name in ("architecture", "sizing"):
按固定顺序依次遍历 architecture 和 sizing，name 在两次循环中取不同值。

L022             context[name] = agents[name].run(context)
调用对应 Agent，把当前 context 发给模型；返回字典存入 context[name]。第二次 sizing 因此可以看到第一次的架构结果。

L023             save(name+".json", context[name])
保存对应角色输出为 architecture.json 或 sizing.json，不改变模型回答里的字段。

L024             print(f"{name}: OK", flush=True)
终端打印该角色已完成；flush=True 立即刷新输出，避免长任务中提示被缓冲。

L025             params = context["sizing"]["parameters"]
从尺寸回答中取出 parameters 字典，作为当前迭代参数 params；初始 rationale 仍在 context 中保留。

L026             # Initial evaluation plus up to max_iterations actual parameter updates.
注释说明循环语义：初始评估一次，之后最多 max_iterations 次参数更新。注释本身不执行。

L027             for step in range(cfg["max_iterations"]+1):
range(2+1) 产生 0、1、2；每个 step 对应一次评估。Python range 不包含右端点。

L028                 step_dir = out / f"iteration-{step}"
构造本轮子目录名 iteration-0、iteration-1 等。

L029                 step_dir.mkdir()
实际创建本轮目录，供网表文件保存。

L030                 plan = agents["simulation"].run(**context, "parameters": params)
调用 simulation Agent，传入 context 展开的字段和当前 parameters。后面的 parameters 字段是迭代当前值，不是初始 sizing 记录。

L031                 result = MockSimulation().run(params, cfg["specification"], plan["analyses"],
step_dir)
创建 MockSimulation 工具并立即运行；四个输入是当前尺寸、目标指标、模型选出的分析列表和本轮输出目录；返回 Mock 指标字典 result。

L032                 record = {"iteration": step, "parameters": params, "simulation_plan": plan, "
simulation": result}
把迭代编号、当前参数、分析计划和计算结果组装成 record 字典，形成可追踪的单轮记录。

L033                 history.append(record)
把 record 对象放入 history。这里保留对象引用，后面给 record 添加优化建议时，同一 history 元素也会变化。

L034                 save("history.json", history)
把截至本轮的完整 history 写入 history.json，在问优化角色之前先保留已完成评估。

L035                 print(f"simulation {step}: MOCK {result['metrics']}", flush=True)
在终端明确打印 MOCK 和当前指标，避免把它误认为实测电路结果。

L036                 if step == cfg["max_iterations"]:
判断当前轮是不是允许的最后一轮评估。达到上限时不再问优化 Agent。

L037                     break
从最近的 for step 循环跳出；后面的保存 summary 仍会继续执行。

L038                     advice = agents["optimization"].run(**context, **record)
请求 optimization Agent；把指标/初始设计 context 与当前 record 展开成一个输入字典。默认没有传整个 history。

L039                     record["optimization"] = advice
将模型优化建议加入当前 record，包含完整下一轮参数、理由和 stop；history 中该轮记录随之更新。

L040                     save("history.json", history)
再次保存 history.json，这次把刚获得的优化建议也写进去。

L041                     if advice["stop"] and result["all_targets_met_synthetically"]:
and 要求模型建议停止和 Python 的合成目标判断同时为真，才走下面 break；仅模型声称完成不够。

```

```
L042                 break
```

两个条件都满足时提前离开迭代循环。此时本次 advice 的新参数没有再被应用或评估。

```
L043                 params = advice["parameters"]
```

否则把优化输出参数指定为下一轮 params，然后循环回到本轮 for 开头；没有原地改旧字典，也没有比较建议是否变好。

```
L044                 save("summary.json", {"status": "completed", "llm_backend": "local_qwen", "
simulation_backend": "mock",
```

循环结束开始写 summary.json：completed 表示流程走完，llm_backend 标注本地 Qwen，simulation_backend 标注 mock。

```
L045                 "real_circuit_validated": False, "evaluations": len(history), "
final": history[-1]})
```

继续 summary 字典：真实电路未验证，评估次数是 history 长度，final 取最后一条记录；这里没有搜索“最佳”记录。

```
L046             except Exception as exc:
```

捕获 try 中发生的大多数普通异常，命名为 exc；不会捕获所有系统级终止情形，例如 KeyboardInterrupt 不属于 Exception。

```
L047                 save("failure.json", {"status": "failed", "error": str(exc)})
```

在实验目录写 failure.json，记录失败状态和错误文字。若文件系统自身无法写入，这次记录也可能失败。

```
L048                 raise
```

重新抛出原异常，使终端能看到调用栈并让进程失败退出，而不是打印失败后继续伪装成功。

```
L049             print(f"Results: {out}")
```

正常完成后打印结果目录，便于用户找到此次实验的产物。

```
L050     [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L051 if __name__ == "__main__":
```

当模块通过 -m 直接执行时 __name__ 为 __main__；被其他文件 import 时通常不是，因此不会自动跑实验。

```
L052     main()
```

调用上面定义的 main()，这是控制权进入正式工作流的具体位置。

A12. analog_agents/agents.py

定义角色允许返回的数据结构，以及所有角色共享的 Agent 类。SCHEMAS 的键名必须和 prompts 文件名、run.py 中的角色名称对应。

L001 from .config import project_path

从同一包的 config.py 导入路径检查函数。开头 . 表示相对当前 analog_agents 包，不是磁盘当前工作目录。

L002 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L003 def obj(properties):

定义 obj 辅助函数，接收一个“字段名 -> 字段规则”的字典，组装对象类型的 JSON Schema。

L004 return {"type": "object", "properties": properties, "required": list(properties), "additionalProperties": False}

返回 Schema : type=object 要求字典；properties 描述各字段；required=list(properties) 将所有字段名设为必填；additionalProperties=False 拒绝未声明字段。

L005 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L006 TEXT = {"type": "string"}

TEXT 是字符串字段的通用规则，供 rationale 等字段复用；不是提示词正文。

L007 PARAMETERS = obj({

调用 obj，开始建立六个电路参数的共同规则 PARAMETERS；后面的字典是传给 obj 的参数。

L008 "input_w_um": {"type": "number", "minimum": 1, "maximum": 200},

input_w_um 为输入对晶体管宽度，单位微米；必须是数值，范围 1—200。number 可含小数，不限整数。

L009 "input_l_um": {"type": "number", "minimum": 0.18, "maximum": 5},

input_l_um 为沟道长度，范围 0.18—5 微米；当前概念模板把它用于所有 MOS，不是只有输入对。

L010 "load_w_um": {"type": "number", "minimum": 1, "maximum": 400},

load_w_um 为 PMOS 镜像负载宽度，范围 1—400 微米。

L011 "stage2_w_um": {"type": "number", "minimum": 1, "maximum": 400},

stage2_w_um 为第二级 PMOS 宽度，范围 1—400 微米。

L012 "bias_ua": {"type": "number", "minimum": 1, "maximum": 300},

bias_ua 为演示偏置电流，范围 1—300 微安；网表尾电流使用它，第二级理想电流源使用它的两倍。

L013 "compensation_pf": {"type": "number", "minimum": 0.1, "maximum": 20}

compensation_pf 为补偿电容，范围 0.1—20 皮法；大于零也避免 Mock 带宽计算除以零。

L014 })

} 结束六字段字典，) 结束 obj 调用；其返回值赋给 PARAMETERS。

L015 SCHEMAS = {

建立 SCHEMAS 字典，以角色名字索引各自输出规则。

L016 "architecture": obj({"topology": {"type": "string", "enum": ["two_stage_cmos_opamp"]}, "rationale": TEXT, "assumptions": {"type": "array", "items": TEXT}}),

architecture 必须返回 topology、rationale、assumptions。topology 用 enum 只允许 two_stage_cmos_opamp；assumptions 是字符串数组。当前不接受任意 ADC 架构名。

L017 "sizing": obj({"parameters": PARAMETERS, "rationale": TEXT}),

sizing 必须返回 parameters 和 rationale；parameters 复用上面的六参数规则。

L018 "simulation": obj({"analyses": {"type": "array", "items": {"type": "string", "enum": ["op", "ac", "tran"]}, "minItems": 1}, "rationale": TEXT}),

simulation 必须返回 analyses 与 rationale。analyses 至少一个元素，每项仅允许 op/ac/tran；这里没有 uniqueItems，所以规则本身不禁止重复分析名称。


```
L019         "optimization": obj({"parameters": PARAMETERS, "rationale": TEXT, "stop": {"type": "boolean"}})
```

optimization 必须返回完整 parameters、rationale 和布尔 stop。即使只改一个参数，也要把另外五项返回。

```
L020     }
```

结束 SCHEMAS 字典定义；此前都是规则数据，没有向模型请求设计。

```
L021     [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L022     class Agent:
```

定义统一 Agent 类，不为四个角色各复制一套网络代码。

```
L023         def __init__(self, name, client):
```

构造 Agent 时接收角色名字 name 和已建立的客户端 client。

```
L024             self.name, self.client = name, client
```

同时保存名字和客户端到 self.name、self.client，后面据此找提示词和 Schema。

```
L025             self.prompt = project_path(f"prompts/{name}.md").read_text()
```

把名字填进 prompts/{name}.md，从项目目录读取整段文字，存到 self.prompt。比如 sizing 对应 prompts/sizing.md；这里只读一次。

```
L026     [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L027         def run(self, context):
```

定义 run 方法，参数 context 是该次要让角色看到的任务资料字典。

```
L028             return self.client.ask(self.prompt, context, SCHEMAS[self.name])
```

把角色文字、上下文和对应 Schema 传给 LocalClient.ask，并把返回字典原样交回。Agent 类自身不写结果文件、不运行仿真器。

A13. analog_agents/client.py

所有角色共用的请求实现。负责连接本地 vLLM、构造消息、解析并校验 JSON；本文件不直接调用 torch 或 GPU。

L001 import json

导入 json，用于把请求字典转成文本，以及把模型返回文本解析成字典。

L002 from openai import OpenAI

从已安装 openai 包导入 OpenAI 客户端类；包名不表示一定访问云端，实际目标由 base_url 决定。

L003 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L004 class LocalClient:

定义 LocalClient 类，封装本地模型通信。实例化后可以重复调用 ask。

L005 def __init__(self, config):

构造函数 __init__ 在 LocalClient(cfg) 时执行；self 是新客户端对象，config 是传进来的配置字典。

L006 self.config = config

保存配置引用为 self.config，后面的请求会从这里取模型名和生成设置。

L007 self.api = OpenAI(base_url=f"http://127.0.0.1:{config['port']}/v1",

创建 SDK 客户端并保存在 self.api。f 字符串把 port 替换进本机 URL；末尾 /v1 是 API 基础路径，此时通常还没有发送推理请求。

L008 api_key="local-only", timeout=config["timeout_seconds"], max_retries=1)

继续上行构造：local-only 是占位 key，timeout 使用配置值；max_retries=1 允许某些请求失败后再试一次。右括号结束 OpenAI(...) 调用。

L009 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L010 def ask(self, prompt, payload, schema):

定义 ask 方法，三个输入分别为角色提示词 prompt、任务资料 payload、输出格式规则 schema。

L011 response = self.api.chat.completions.create(

调用 SDK 的聊天生成方法，真正向服务发请求并等待结果，返回对象命名为 response；后面缩进续行都是本次调用的参数。

L012 model=self.config["served_model"],

将模型 API 别名传给 model，不是远程仓库路径；必须与服务 --served-model-name 一致。

L013 messages=[{"role": "system", "content": prompt + "\nReturn only a JSON object conforming to this schema: " + json.dumps(schema)},

建立消息列表第一项 system：角色提示词加一句要求 JSON 的说明，再加 schema 的 JSON 文本。反斜线 n 表示换行；这里把规则也写进模型可见文字。

L014 {"role": "user", "content": json.dumps(payload, ensure_ascii=False)}],

第二项 user 保存本次上下文 payload 的 JSON 文本；ensure_ascii=False 保留中文，而不是写成 Unicode 转义。方括号结束两条消息列表。

L015 temperature=self.config["temperature"], max_tokens=self.config["max_tokens"],

temperature 设置采样随机程度，max_tokens 限制这次最多生成多少 token；两项从集中配置读取。

L016 extra_body={"chat_template_kwargs": {"enable_thinking": False}, "guided_json": schema})

extra_body 是扩展字段：enable_thinking=False 控制 Qwen3 模板，guided_json 将 Schema 传给 vLLM 的结构化生成机制。右括号结束请求，程序等待它返回。

L017 choice = response.choices[0]

取 choices 列表第 0 个候选答案；默认请求只需要一个候选，不是在四个 Agent 中选第一个。

```
L018         if choice.finish_reason != "stop":
```

检查生成结束原因是不是 stop。此处字符串 stop 与优化 JSON 中的布尔 stop 不是同一个变量。

```
L019         raise ValueError(f"Incomplete model output: {choice.finish_reason}")
```

如果输出因长度等原因中断，抛 ValueError 并带出原因，避免把截断 JSON 当成完成结果。

```
L020         result = json.loads(choice.message.content)
```

把模型消息正文从字符串解析为 Python 字典。若正文包含非 JSON 文本或损坏格式，这一步会抛 JSONDecodeError。

```
L021         from jsonschema import validate
```

导入 jsonschema 库的 validate 函数，准备在客户端再次检查响应；该导入位于函数内部，执行到这里才发生。

```
L022         validate(result, schema)
```

检查 result 是否满足这个 Agent 的 Schema：字段名、类型、范围等。失败会抛 ValidationError，不会自动修复或再次问模型。

```
L023         return result
```

校验成功后返回 result 给 Agent.run，再返回给主流程。没有在这里保存响应文件；保存由 run.py 或测试脚本负责。

A14. prompts/architecture.md

纯文本角色提示词，由 Agent.__init__ 读取。本文件只有一条物理长行，在 PDF 中会折行显示。

```
L001 You are the Architecture Agent. Propose a two-stage CMOS operational amplifier with an NMOS differential pair, PMOS current mirror active load, PMOS common-source second stage, and Miller compensation. Bias sources are ideal in this first demo; transistor-level bias generation is future work. Explain assumptions in concise Chinese. No PDK is available: call this a conceptual architecture, not a validated design. Respect the supplied specification.
```

整行作为 system 提示词的一部分送给模型：要求两级运放、NMOS 输入对、PMOS 镜像和 PMOS 第二级、Miller 补偿；承认偏置源理想化和没有 PDK；用简洁中文说明假设。该行不创建电路，也不改变 Python 固定拓扑。

A15. prompts/sizing.md

纯文本角色提示词，由 Agent.__init__ 读取。本文件只有一条物理长行，在 PDF 中会折行显示。

```
L001 You are the Sizing Agent. Supply initial transistor geometry, bias current and compensation capacitance for the proposed two-stage CMOS opamp. Units are micrometers, microamperes and picofarads as indicated by field names. The common channel length applies to all devices in the demo template. Geometry without a foundry PDK is illustrative only. Explain your reasoning concisely in Chinese. Explicitly say these initial guesses are unverified; never claim they ensure or meet any actual circuit performance target. Start near input_w_um=20, input_l_um=1, load_w_um=40, stage2_w_um=80, bias_ua=40, compensation_pf=2. The optimizer will adjust the initial guess after receiving MOCK metrics.
```

整行说明尺寸角色要输出几何尺寸、偏置、电容，单位由字段名决定；共同沟道长度用于所有器件；必须说初值未验证，不能声称已保证指标；建议从 20/1/40/80/40/2 附近起步，后面由 Mock 与优化迭代。它只是模型指令，不是参数的硬编码赋值。

A16. prompts/simulation.md

纯文本角色提示词，由 Agent.__init__ 读取。本文件只有一条物理长行，在 PDF 中会折行显示。

```
L001 You are the Simulation Agent. Choose op, ac and/or tran analysis for the supplied two-stage opamp and explain in concise Chinese. The program will render a fixed safe netlist template and invoke a MockSimulation tool. MOCK results are synthetic and do not constitute SPICE or silicon validation. Do not invent executed simulations, PDK models or measured data. Do not output shell commands.
```

整行要求从 op/ac/tran 中选分析并用中文说明；明确 Python 使用固定模板和 MockSimulation，不能声称真正执行 SPICE，不得输出 shell 命令。真正允许字段还受 agents.py 的 Schema 限制。

A17. prompts/optimization.md

纯文本角色提示词，由 Agent.__init__ 读取。本文件只有一条物理长行，在 PDF 中会折行显示。

L001 You are the Optimization Agent. Read the provided MOCK metrics and targets, then propose bounded updated parameters, explain the changes concisely in Chinese, and set stop only if the workflow should stop. MOCK is a synthetic workflow demonstration, not real circuit optimization. Use the provided mock_formula for this demonstration. Improve deficient targets, avoid unnecessary power increase. Return the complete updated parameters including unchanged values.

整行要求读取 Mock 指标和目标，利用给出的合成公式给出受限参数更新、中文理由和停止建议；要求完整返回参数；不把合成优化说成真实电路设计。Python 最终仍会检查数值范围与停止条件。

A18. analog_agents/simulation.py

工具接口与合成仿真实现。模型只提出分析计划，本文件用 CPU 生成概念网表并计算人为公式；不存在真实仿真器子进程。

L001 `"""Explicitly synthetic backend. No subprocess executes model-generated code."""`

模块文档字符串明确说明合成后端和“不执行模型生成代码”的边界，不是程序指令。

L002 `import math`

导入标准数学库，主要使用以 10 为底的对数 $\log_{10}$ 。

L003 `from typing import Protocol`

导入 Protocol，用来声明未来仿真工具应有的方法形状；它不是一个实际仿真器。

L004 `from jsonschema import validate`

导入 JSON Schema 校验函数，保证传进来的尺寸有完整字段、正确类型且在允许范围内。

L005 `from .agents import PARAMETERS`

从 agents.py 复用 PARAMETERS，避免 Agent 和工具各自定义不同的参数边界。

L006 `【空行】`

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L007 `MOCK_FORMULA = "gain_db=45+12*log10(input_w_um/input_l_um); ugb_mhz=0.8*bias_ua/compensation_pf; phase_margin_deg=min(89,45+12*compensation_pf-0.2*load_pf); power_mw=vdd_v*3*bias_ua/1000. Synthetic only; not physical predictions."`

把 Mock 公式保存为文字，随结果传给优化 Agent 参考。这一字符串本身不参与计算；真正计算在下面 metrics 中，改公式时要同时维护两处。

L008 `【空行】`

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L009 `class Simulator(Protocol):`

定义 Simulator 协议类型，表示兼容的工具应提供 run 方法；当前 MockSimulation 没有显式继承它，也没有强制运行时注册。

L010 `def run(self, parameters, spec, analyses, output_dir) -> dict: ...`

声明 run 需要哪些参数以及返回 dict；... 是省略号占位符，表示这里没有真实算法，不会自动调用仿真器。

L011 `【空行】`

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L012 `def netlist(p, spec, analyses):`

定义 netlist 函数，输入 p 是尺寸字典、spec 是目标/供电信息、analyses 是所选分析名称；输出电路文本。

L013 `validate(p, PARAMETERS)`

先检查 p 是否满足 PARAMETERS，拒绝缺字段、字符串冒充数字或不合理范围。

L014 `# Fixed connectivity, numeric parameters only; illustrative LEVEL=1 models are not a PDK.`

注释强调固定连接模板、只插入数字、LEVEL=1 模型不是工艺 PDK，提醒不可把它当作可靠器件模型。

L015 `w, l, load, stage, bias, cc = [p[k] for k in PARAMETERS["properties"]]`

按 PARAMETERS 中字段插入顺序读取六个值，解包成 w/l/load/stage/bias/cc。改字段顺序或数量会影响这里，因此正文给出了按字段名逐项读取的替换方法。

L016 `commands = {"op": ".op", "ac": ".ac dec 50 1 1G", "tran": ".tran 1n 10u"}`

把分析名字映射成 SPICE 文本：op 是工作点，ac 是从 1 Hz 到 1 GHz 每十倍频程 50 点，tran 是示意 1 ns 步长到 10 us。这里仅生成字符串。

L017 `return f"""* CONCEPTUAL TEMPLATE ONLY - NOT EXECUTED; no foundry PDK`

开始返回多行 f 字符串，花括号里的 Python 表达式会换成实际值；首行 * 是 SPICE 注释，明确未执行、无 PDK。

L018 `* NMOS pair + PMOS mirror + PMOS common source second stage`

仍在多行字符串内，是网表注释，说明 NMOS 输入对、PMOS 镜像和 PMOS 第二级；不是 Python 乘法。

L019 VDD vdd 0 {spec['vdd_v']}

网表电压源 VDD 从 vdd 节点到地 0，电压取 spec 的 vdd_v。

L020 VINP inp 0 DC {spec['vdd_v']/2} AC 0.5

正输入电压源的直流共模是供电一半，AC 幅度 0.5；只是概念网表的激励定义。

L021 VINN inn 0 DC {spec['vdd_v']/2} AC 0.5 180

负输入电压源同样共模，AC 幅度 0.5 且相位 180 度，构成一对反相小信号输入。

L022 ITAIL tail 0 {bias}u

从 tail 到地的理想尾电流源，数值为 bias 微安。没有使用真实晶体管偏置网络。

L023 M1 n1 inp tail 0 NM W={w}u L={l}u

M1 是输入对第一管，节点顺序漏/栅/源/体为 n1、inp、tail、0，模型 NM；W 和 L 插入微米数值。

L024 M2 n2 inn tail 0 NM W={w}u L={l}u

M2 是输入对第二管，漏极 n2、栅极 inn，其余与第一管类似。

L025 M3 n1 n1 vdd vdd PM W={load}u L={l}u

M3 是二极管连接 PMOS，漏/栅都为 n1，源/体为 vdd；load 是其宽度。

L026 M4 n2 n1 vdd vdd PM W={load}u L={l}u

M4 是 PMOS 镜像另一支路，漏在 n2、栅在 n1，复制镜像偏置关系；这里没有计算镜像精度。

L027 M5 out n2 vdd vdd PM W={stage}u L={l}u

M5 是第二级 PMOS 共源管，漏为 out、栅为 n2、源/体为 vdd，宽度取 stage。

L028 IBIAS out 0 {2*bias}u

第二级输出到地的理想电流源，设置为两倍 bias 微安；这是演示假设，不是优化算法推导出的完整偏置设计。

L029 CC n2 out {cc}p

补偿电容 CC 接在第一级输出 n2 与最终输出 out 之间，单位皮法。

L030 CL out 0 {spec['load_pf']}p

负载电容 CL 从输出到地，使用 specification.load_pf。

L031 .model NM NMOS LEVEL=1 VT0=0.45 KP=120u LAMBDA=0.04

定义名为 NM 的简化 NMOS LEVEL=1 模型；阈值 0.45、KP=120u、LAMBDA=0.04 是示例数值，不是代工厂参数。

L032 .model PM PMOS LEVEL=1 VT0=-0.45 KP=50u LAMBDA=0.04

定义 PM 的简化 PMOS 模型；负阈值和不同 KP 体现符号/类型差异，依旧不是工艺验证模型。

L033 "" + "\n".join(commands[a] for a in analyses) + "\n.end\n"

结束多行模板；依 analyses 逐项取 commands，以换行连接并附加 .end。return 整段字符串。没有调用 ngspice，也没有执行 .op/.ac/.tran。

L034 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L035 class MockSimulation:

定义 MockSimulation 类，提供当前实际被 run.py 使用的工具。

L036 def run(self, parameters, spec, analyses, output_dir):

定义工具 run 方法，接收尺寸、规格、分析列表、输出目录。

L037 validate(parameters, PARAMETERS)

在工具入口再次校验尺寸；即使将来有人绕开 Agent 直接调用工具，也会有这道检查。

L038 (output_dir / "conceptual_not_executed.cir").write_text(netlist(parameters, spec, analyses))

调用上面的 netlist 生成文字，并写成当前轮目录的 conceptual_not_executed.cir。工具随后不读取这个文件来计算指标。

L039 p = parameters

为 parameters 起一个较短别名 p；没有复制数据，也没有修改参数。

L040 metrics = {"gain_db": round(45+12*math.log10(p["input_w_um"]/p["input_l_um"]), 3),
开始 metrics 字典, gain_db 使用 $45+12\log_{10}(W/L)$ 的合成公式, round(...,3) 保留三位小数。这里只是教学映射,
不是晶体管小信号推导。

L041 "ugb_mhz": round(0.8*p["bias_ua"]/p["compensation_pf"], 3),
合成带宽为 $0.8\cdot\text{偏置电流}/\text{补偿电容}$, 保存为 ugb_mhz, 保留三位小数。

L042 "phase_margin_deg": round(min(89,
 $45+12\cdot p[\text{"compensation_pf"}]-0.2\cdot\text{spec}[\text{"load_pf"}]$), 3),
合成相位裕度为 $45+12\cdot\text{补偿电容}-0.2\cdot\text{负载电容}$, 并用 min 限制最大 89 度; 保留三位小数。

L043 "power_mw": round(spec["vdd_v"]*3*p["bias_ua"]/1000, 4)}
合成功耗为供电*3*偏置/1000, 单位毫瓦, 保留四位小数; 闭合 metrics 字典。

L044 checks = {k: metrics[k] >= spec[k+"_min"] for k in ("gain_db", "ugb_mhz", "
phase_margin_deg")}
对增益、带宽和相位裕度三个指标, 逐个比较 metrics[k] 是否大于等于 spec[k+"_min"], 产生真/假的 checks 字典。

L045 checks["power_mw"] = metrics["power_mw"] <= spec["power_mw_max"]
功耗是越小越好, 因此单独与 power_mw_max 用 <= 比较, 而不是沿用前三项的 >=。

L046 return {"backend": "mock", "synthetic": True, "real_spice_executed": False,
开始返回结果字典, 明确 backend=mock、synthetic=True、real_spice_executed=False, 阻止来源被误解。

L047 "warning": "仅用于软件流程测试; 不得作为真实电路性能或流片依据。",
附带中文警示说明这些数字只适合软件流程测试, 不能用于真实电路或流片判断。

L048 "metrics": metrics, "checks": checks, "all_targets_met_synthetically":
all(checks.values()),
附上指标、每项检查及 all(checks.values()); all 只有在每个布尔值都为真时才返回真。

L049 "mock_formula": MOCK_FORMULA}
把上面保存的公式文字也交回, 让优化角色知道它正在优化什么合成规则; 结束返回字典。

A19. scripts/stop.sh

日常 Bash 入口。使用绝对的虚拟环境 Python 路径, 即使没有手动激活也不会意外选中系统 Python; 仍需先启动模型才能调用 API。

L001 #!/usr/bin/env bash
shebang 声明这是 Bash 文件; 日常用 bash 文件名 显式解释。

L002 set -euo pipefail
set -e 让通常的命令失败中断脚本, -u 拒绝未定义变量, pipefail 使管道中间失败也可传播; Bash 某些条件语境有例外, 不能看作完整异常处理。

L003 source "\${dirname -- "\$0"}/env.sh"
根据脚本文件位置 source 同目录 env.sh, 统一 PROJECT_ROOT 和缓存/临时环境。

L004 cd "\$PROJECT_ROOT"
把当前子进程工作目录切到 PROJECT_ROOT, 使后续 scripts/... 相对路径有确定含义。

L005 exec "\$PROJECT_ROOT/.venv/bin/python" scripts/service.py stop
用 exec 执行项目 Python 的 scripts/service.py, 并传入 stop; 只停止状态文件记录的项目进程组。

A20. scripts/download_model.py

安装或主动更新模型时执行。日常启动模型与运行实验不会自动调用它。需要网络，权重与下载记录都位于项目目录。

```
L001 import json
```

导入 json，用于保存模型下载记录。

```
L002 from huggingface_hub import snapshot_download
```

从 Hugging Face Hub 包导入 snapshot_download，它能按仓库快照下载匹配的文件。

```
L003 from analog_agents.config import load_config, project_path
```

导入集中配置读取和项目路径边界检查。

```
L004 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L005 c = load_config()
```

读取 settings.json，获得模型仓库、固定修订和本地目录；此脚本没有 main 保护，import 它也会执行下载逻辑，因此不要随意导入。

```
L006 path = snapshot_download(c["model_repo"], revision=c["model_revision"],
```

调用 snapshot_download，按 model_repo 和 model_revision 选择远程仓库的固定快照。

```
L007     local_dir=str(project_path(c["model_dir"])), max_workers=2,
```

local_dir 显式指定项目内模型目录；max_workers=2 限制并行下载工作线程，不是使用两张 GPU。

```
L008     allow_patterns=["*.json", "*.safetensors", "*.txt", "*.jinja", "LICENSE", "README.md"])
```

仅下载列出的文件模式，包含配置、safetensors 权重、分词文本/模板和许可证说明；* 是文件名通配符。结束下载调用，函数返回本地路径。

```
L009 project_path("logs/model_download.json").write_text(json.dumps({"path": path, "repo":
```

```
c["model_repo"], "revision": c["model_revision"], "status": "complete"}, indent=2))
```

只有上面下载正常返回才写 complete 记录到 logs/model_download.json，包含路径、仓库、修订。下载抛错时不会执行这一行。

```
L010 print(path)
```

打印模型文件所在路径，供用户确认；这不代表模型已加载到 GPU。

A21. scripts/install.sh

可重复安装的辅助脚本，不是日常运行入口。它优先同步 requirements.lock，最后下载模型；会联网但只针对项目环境。

```
L001 #!/usr/bin/env bash
```

声明 Bash 脚本。

```
L002 set -euo pipefail
```

启用常见错误时退出、未定义变量报错、管道失败传播，避免安装一半后忽略明显错误继续运行。

```
L003 source "${dirname -- "$0"}/env.sh"
```

先 source env.sh，确保下载、缓存、临时目录都指向项目内。

```
L004 cd "$PROJECT_ROOT"
```

切换到项目根目录，使相对环境和 requirements 路径稳定。

```
L005 if [[ ! -x .runtime/tools/bin/uv ]]; then
```

检测项目 uv 文件是否不是可执行文件，只有缺失时才做下一步引导安装。

```
L006     python3 -m pip install --isolated --no-cache-dir --target ".runtime/tools"
uv==0.8.22
```

用已有 python3 的 pip 把 uv 0.8.22 安装到 .runtime/tools。--target 限制安装目的地，--no-cache-dir 关闭 pip 下载缓存，--isolated 忽略用户 pip 配置与环境设置；临时目录仍由程序的环境决定。

```
L007 fi
```

fi 结束 uv 是否需要安装的条件分支。

```
L008 .runtime/tools/bin/uv python install 3.11.13
```

用项目 uv 确保 Python 3.11.13 已安装；其目录由 UV_PYTHON_INSTALL_DIR 指定。

```
L009 if [[ ! -d .venv ]]; then
```

检查 .venv 目录是否不存在；已有环境时不会重新创建。

```
L010     .runtime/tools/bin/uv venv --python 3.11.13 --seed .venv
```

建立使用指定 Python 的 .venv，并用 --seed 提供 pip 等基础包；不是修改系统 Python。

```
L011 fi
```

结束虚拟环境创建条件。

```
L012 if [[ -f requirements.lock ]]; then
```

检查项目是否有完整 requirements.lock 文件。

```
L013     .runtime/tools/bin/uv pip sync --python .venv/bin/python requirements.lock
```

有锁文件就用 uv pip sync 使项目环境与锁定列表同步；可能移除未列出的包，因此不能把它理解为仅添加缺失依赖。

```
L014 else
```

如果没有锁文件，进入备用安装方式。

```
L015     .runtime/tools/bin/uv pip install --python .venv/bin/python -r requirements.txt
```

按 requirements.txt 安装主要依赖，再由解析器选择其间接依赖；没有完整锁文件时可重复性较弱。

```
L016 fi
```

结束锁文件选择分支。

```
L017 .venv/bin/python scripts/download_model.py
```

用项目 Python 执行 download_model.py，下载配置中指定模型；安装脚本不会自动启动 vLLM 或运行 Agent。

A22. scripts/test_agents.py

把相同固定资料分别发送给四个角色，验证每个角色能独立响应。它不是完整端到端编排，合成资料也不是实际仿真。

L001 """Calls each agent independently against the real local Qwen API."""

模块说明：每个角色都调用真实的本地 Qwen API，而不是使用假模型响应。

L002 import json

导入 json，用于保存测试回答。

L003 from analog_agents.config import load_config, project_path

导入集中配置读取与项目路径检查。

L004 from analog_agents.agents import Agent

导入 Agent 类，使用与正式系统相同的角色实现。

L005 from analog_agents.client import LocalClient

导入 LocalClient 类，用统一 API 路径请求服务。

L006 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L007 def main():

定义主函数，避免被导入时自动测试。

L008 c = load_config()

读取当前配置，确定本地模型名、端口及设计规格。

L009 params = dict(input_w_um=20, input_l_um=1, load_w_um=40, stage2_w_um=80, bias_ua=40, compensation_pf=2)

建立固定六参数示例字典；dict(key=value) 是 Python 建字典的一种写法。

L010 payload = {"specification": c["specification"], "architecture": {"topology": "two_stage_cmos_opamp"},

开始 payload 输入，包含配置中的目标和固定两级运放拓扑标签。

L011 "parameters": params, "simulation": {"backend": "mock", "synthetic": True,

继续输入，加入参数以及明确 backend=mock、synthetic=True 的合成结果标记。

L012 "metrics": {"gain_db": 60.6, "ugb_mhz": 16, "phase_margin_deg": 68, "power_mw": 0.216}}}

手工放入示例指标，闭合 metrics、simulation、payload 三层字典；这里没有运行 MockSimulation 重新计算。

L013 results = {}

建立空的 results 字典，逐个保存角色测试结果。

L014 for name in ("architecture", "sizing", "simulation", "optimization"):

按固定四个角色循环；若将来新增角色，这个列表不会自动更新。

L015 results[name] = {"status": "passed", "response": Agent(name, LocalClient(c)).run(payload)}

为当前角色创建客户端和 Agent，真正调用 run(payload)。只有 run 正常返回时整条赋值才成功，才记录 status=passed。

L016 project_path("logs/agent_tests.json").write_text(json.dumps(results, ensure_ascii=False, indent=2))

每通过一个角色就把累计结果保存到 logs/agent_tests.json；如果后续角色失败，文件可能只有已通过的一部分，不能单看存在就认为四个都成功。

L017 print(name, "PASS", flush=True)

立即打印当前角色 PASS，方便长时间测试时查看进度。

L018 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L019 if __name__ == "__main__":

主程序入口保护，import 时不执行 main。

```
L020      main()
```

调用 main，开始四次独立角色测试。

A23. tests/test_workflow.py

pytest 离线测试。它使用 CPU 和 Mock 工具，不调用模型 API。运行时用项目内 --basetemp 指定测试临时目录。

L001 import pytest

导入 pytest 测试框架。

L002 from jsonschema import ValidationError

导入 ValidationError，供断言某些非法输入必须报错。

L003 from analog_agents.config import project_path, load_config

导入项目路径和配置读取函数，这是测试对象的一部分。

L004 from analog_agents.simulation import MockSimulation

导入 MockSimulation，测试合成工具行为。

L005 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L006 PARAMS = dict(input_w_um=20, input_l_um=1, load_w_um=40, stage2_w_um=80, bias_ua=40, compensation_pf=2)

固定一套六参数测试输入，所有测试引用它；后面的修改输入用新字典避免改变这个共享样本。

L007 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L008 def test_path_escape():

定义名称以 test_ 开头的测试函数，pytest 会自动发现并调用。

L009 with pytest.raises(ValueError):

声明内部代码必须抛 ValueError；如果没有抛出，pytest 会把本项标为失败。

L010 project_path("../tmp/escape")

尝试访问项目外 ../tmp/escape，应该被 project_path 拒绝。它不会真正写该外部路径。

L011 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L012 def test_synthetic_provenance(tmp_path):

定义合成来源标记测试；tmp_path 是 pytest 提供的该测试临时目录对象。

L013 result = MockSimulation().run(PARAMS, load_config()["specification"], ["op", "ac"], tmp_path)

调用 MockSimulation，使用真实配置中的规格、固定参数以及 op/ac 分析列表，文件写到测试临时目录。

L014 assert result["synthetic"] and not result["real_spice_executed"]

断言结果明确是 synthetic 且 real_spice_executed 为假，防止把 Mock 标成真实仿真。

L015 assert result["metrics"]["power_mw"] == pytest.approx(0.216)

对固定输入，合成功耗应接近 0.216；approx 允许合理浮点误差。

L016 assert "NOT EXECUTED" in (tmp_path / "conceptual_not_executed.cir").read_text()

读取生成网表，要求存在 NOT EXECUTED 字样，确保产物也有未执行标记。

L017 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L018 def test_reject_bad_sizing(tmp_path):

定义非法尺寸输入测试。

L019 with pytest.raises(ValidationError):

预期工具会抛出 jsonschema 的 ValidationError。

L020 MockSimulation().run(**PARAMS, "compensation_pf": 0, load_config()["specification"], ["op"], tmp_path)

复制固定参数并把 compensation_pf 覆盖为 0；Schema 下限 0.1 应拒绝它，而不是发生除零或静默接受。

L021 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L022 `def test_parameter_update_changes_evaluation(tmp_path):`

定义参数更新会改变计算结果的测试。

L023 `tool = MockSimulation()`

创建 MockSimulation 工具实例。

L024 `first = tool.run(PARAMS, load_config()["specification"], ["ac"], tmp_path)`

先以原始参数运行一次得到 first。

L025 `second = tool.run(**PARAMS, "bias_ua": 80, load_config()["specification"], ["ac"], tmp_path)`

再用新字典把 bias_ua 改为 80，其他参数相同，运行得到 second。

L026 `assert second["metrics"]["ugb_mhz"] == 2 * first["metrics"]["ugb_mhz"]`

合成带宽公式与偏置成正比，所以在此固定电容下第二次带宽应是第一次两倍。这验证参数确实进入了计算，不证明真实电路比例关系。

A24. pytest.ini

pytest 的项目测试发现设置，避免把 scripts/ 下的日常 API 脚本当成离线测试自动收集。

L001 [pytest]

声明 ini 文件的 pytest 配置段。

L002 `testpaths = tests`

让 pytest 默认只从 tests 目录寻找测试；不会因此运行 scripts/test_qwen.py。

A25. scripts/audit_paths.py

只检查项目生成目录与声明的缓存路径，不遍历整个用户主目录或私人文件。这是配置/链接审计，不是内核系统调用跟踪。

```
L001 """Audit project-owned artifact trees and configured writable locations, not private home files."""
```

模块说明限定检查对象为项目产物和可写路径，不检查私人目录。

```
L002 import json
```

导入 json，以便写入机器可读的审计结果。

```
L003 import os
```

导入 os，用于读取环境变量和遍历目录树。

```
L004 from analog_agents.config import ROOT, project_path
```

导入 ROOT 与 project_path，复用同样的边界判断。

```
L005 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L006 def main():
```

定义主函数，真正执行检查的入口。

```
L007     keys = ['TMPDIR', 'TMP', 'TEMP', 'XDG_CACHE_HOME', 'XDG_CONFIG_HOME', 'XDG_DATA_HOME', 'XDG_STATE_HOME',
```

建立待检查环境变量名列表，先列临时目录和 XDG 目录。

```
L008         'UV_CACHE_DIR', 'UV_PYTHON_INSTALL_DIR', 'UV_PYTHON_BIN_DIR', 'PIP_CACHE_DIR', 'HF_HOME',
```

续行增加 uv、pip、Hugging Face 的路径变量。

```
L009         'TORCH_HOME', 'TORCH_EXTENSIONS_DIR', 'TRITON_CACHE_DIR', 'CUDA_CACHE_PATH', 'VLLM_CACHE_ROOT',
```

续行增加 torch、Triton、CUDA 与 vLLM 缓存变量。

```
L010         'VLLM_CONFIG_ROOT', 'NUMBA_CACHE_DIR', 'MPLCONFIGDIR']
```

续行列出 vLLM 配置、Numba 与 Matplotlib 路径，并结束列表。

```
L011     paths = {key: str(project_path(os.environ[key])) for key in keys}
```

逐个从 os.environ 读变量，调用 project_path 验证仍在项目内，再转换成字符串记录。未 source env.sh 而缺变量会直接报 KeyError。

```
L012     links = []
```

建立空列表 links，用来记录找到的符号链接。

```
L013     trees = ['.venv', '.runtime', 'models', 'scripts', 'config', 'prompts', 'analog_agents', 'tests', 'logs', 'outputs', 'docs']
```

只列本项目创建的目录树，不扫描 .ssh、.aws 或其他私人文件夹。

```
L014     files = 0
```

初始化文件计数器为 0。

```
L015     for tree in trees:
```

逐个处理白名单中的项目目录。

```
L016         for base, dirs, names in os.walk(project_path(tree), followlinks=False):
```

os.walk 遍历该目录；followlinks=False 不沿目录符号链接继续走入潜在在外部树。

```
L017             files += len(names)
```

把当前目录的文件数量累计到总数。

```
L018             for name in dirs+names:
```

对本层目录名和文件名都检查，因为符号链接可以指向目录，也可以指向文件。

```
L019                 path = project_path(base) / name
```

将本层路径和条目名组合成待检查 Path。

```
L020             if path.is_symlink():
只对符号链接进入下方分支。

L021             target = path.resolve()
resolve 得到该链接最终目标，供边界检查。

L022             links.append({'path': str(path), 'target': str(target), 'inside_project':
target.is_relative_to(ROOT)})
保存链接本身路径、目标路径和是否位于 ROOT 内的布尔值。

L023     outside = [item for item in links if not item['inside_project']]
用列表推导式筛出所有目标位于项目外的链接，供审阅。

L024     report = {'status': 'passed' if not outside else 'review_required', 'configured_write_paths':
paths,
开始报告字典：没有外部链接时为 passed，否则 review_required，并附所有配置写入路径。

L025     'artifact_files_checked': files, 'symlinks_checked': len(links), 'external_symlinks':
outside,
续行记录文件总数、链接总数与外部链接详细列表。

L026     'scope': 'Configured paths and project artifact symlinks; not a kernel audit of every
dependency syscall.'}
报告明确审计范围不是依赖的每一次内核系统调用，避免过度声称已经证明绝无外部写入。

L027     project_path('logs/path_audit.json').write_text(json.dumps(report, indent=2))
写入 logs/path_audit.json，便于以后查看。

L028     print(json.dumps(report, indent=2))
同时在终端打印报告。

L029     assert not outside, outside
存在外部链接则断言失败并显示列表，使命令返回失败状态；没有自动删除或改写任何链接。

L030     [空行]
空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L031 if __name__ == '__main__':
主程序入口判断。

L032     main()
调用 main，执行审计。
```

A26. .venv/bin/activate

uv/virtualenv 自动生成的激活代码，scripts/activate.sh 会 source 它。通常不要手工修改；这里只解释当前 Bash 路径及其他 shell 兼容分支。

L001 # Copyright (c) 2020-202x The virtualenv developers
版权注释，标注来自 virtualenv 开发者；不执行任何命令。

L002 #
只有 # 的分隔注释行，没有运行行为。

L003 # Permission is hereby granted, free of charge, to any person obtaining
许可证注释开始：授予获取本软件的人免费使用授权。

L004 # a copy of this software and associated documentation files (the
续行说明授权对象包括软件和相关文档。

L005 # "Software"), to deal in the Software without restriction, including
续行将这些内容统称 Software，并说明可以使用。

L006 # without limitation the rights to use, copy, modify, merge, publish,
续行列举使用、复制、修改、合并、发布等授权。

L007 # distribute, sublicense, and/or sell copies of the Software, and to
续行列举分发、再许可、出售副本等授权。

L008 # permit persons to whom the Software is furnished to do so, subject to
续行说明也可允许接收者这样使用，但须遵守条件。

L009 # the following conditions:
提示后面列出许可证条件；这些都是文字注释。

L010 #
注释段落分隔，不影响脚本执行。

L011 # The above copyright notice and this permission notice shall be
许可证要求保留版权和许可声明。

L012 # included in all copies or substantial portions of the Software.
续行说明完整副本或主要部分中都要保留上述文字。

L013 #
注释段落分隔。

L014 # THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
免责声明：软件按现状提供，没有保证。

L015 # EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
续行说明不提供明示或默示保证。

L016 # MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
续行列出适销性和特定用途适用性等项目。

L017 # NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
续行包含不侵权保证及作者责任说明。

L018 # LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
续行说明作者不承担索赔、损害等责任。

L019 # OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
续行说明包括合同、侵权等不同责任来源。

L020 # WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
续行结束与软件使用有关的免责声明；这一组注释不是 Python/Agent 逻辑。

L021 [空行]
空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L022 # This file must be used with "source bin/activate" *from bash*
注释提醒必须从 Bash 用 source 加载，才能修改当前终端环境。

L023 # you cannot run it directly

注释提醒不能作为普通独立脚本运行后期待父终端生效。

L024 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L025 if [-z "\${SCRIPT_PATH+}"] ; then

判断 SCRIPT_PATH 变量原来是否已被声明；\${变量+} 用来区分未声明与空值，! 对判断取反。

L026 _OLD_SCRIPT_PATH="\${SCRIPT_PATH}"

若存在就保存旧 SCRIPT_PATH，之后恢复，减少对用户终端变量的影响。

L027 fi

fi 结束条件。

L028 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L029 # Get script path (only used if environment is relocatable).

注释说明接下来获取脚本路径，主要为可迁移环境兼容逻辑服务。

L030 if [-n "\${BASH_VERSION:+}"] ; then

检查是否运行于 Bash；\${BASH_VERSION:+} 在其非空时产生 x。

L031 SCRIPT_PATH="\${BASH_SOURCE[0]}"

Bash 下用 BASH_SOURCE[0] 取得当前被 source 的文件路径。

L032 if ["\$SCRIPT_PATH" = "\$0"] ; then

若脚本路径等于 \$0，通常意味着直接运行而不是 source。

L033 # Only bash has a reasonably robust check for source'dness.

注释解释这一直接运行检测是 Bash 专有的相对可靠方法。

L034 echo "You must source this script: \"\$ source \$0" >&2

向标准错误打印必须 source 的提示；>&2 表示输出到错误通道。

L035 exit 33

以退出码 33 终止错误的直接调用，避免用户误以为已激活。

L036 fi

结束直接调用检查。

L037 elif [-n "\${ZSH_VERSION:+}"] ; then

如果不是 Bash，检查是否是 Zsh。

L038 SCRIPT_PATH="\${(%):-%}"

Zsh 分支用该 shell 特有表达式取得文件路径；当前 Bash 工作流不会执行此行。

L039 elif [-n "\${KSH_VERSION:+}"] ; then

再检查是否为 Ksh。

L040 SCRIPT_PATH="\${.sh.file}"

Ksh 分支取脚本路径；当前 Bash 不执行该分支。

L041 fi

结束 shell 类型分支。

L042 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L043 deactivate () {

定义 deactivate shell 函数，之后用户输入 deactivate 会执行其正文；这里只是定义。

L044 unset -f pydoc >/dev/null 2>&1 || true

删除先前定义的 pydoc 函数，丢弃输出；|| true 避免不存在时的失败中断流程。

L045 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L046 # reset old environment variables

注释：下面恢复旧环境变量。

```

L047      # ! [ -z ${VAR+_} ] returns true if VAR is declared at all
注释解释用 ${VAR+_} 检测是否声明变量的技巧。

L048      if ! [ -z "${_OLD_VIRTUAL_PATH:+_}" ] ; then
若保存过旧 PATH 且值非空，执行恢复。

L049          PATH="$_OLD_VIRTUAL_PATH"
把 PATH 恢复为激活之前的搜索路径。

L050          export PATH
export 让恢复后的 PATH 也传给随后启动的子进程。

L051          unset _OLD_VIRTUAL_PATH
删除临时备份变量，避免留下旧状态。

L052      fi
结束 PATH 恢复分支。

L053      if ! [ -z "${_OLD_VIRTUAL_PYTHONHOME+_}" ] ; then
检测是否曾保存 PYTHONHOME，包括原来为空的情况。

L054          PYTHONHOME="$_OLD_VIRTUAL_PYTHONHOME"
恢复原来的 PYTHONHOME 值；它影响 Python 的基础安装查找，与 HOME 不同。

L055          export PYTHONHOME
导出恢复后的 PYTHONHOME。

L056          unset _OLD_VIRTUAL_PYTHONHOME
删除其备份变量。

L057      fi
结束 PYTHONHOME 恢复分支。

L058  [空行]
空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L059      # The hash command must be called to get it to forget past
注释提醒 shell 会缓存命令位置，恢复 PATH 后需要清掉该缓存。

L060      # commands. Without forgetting past commands the $PATH changes
续行说明不清缓存可能仍用之前找到的程序。

L061      # we made may not be respected
续行结束上述说明。

L062      hash -r 2>/dev/null
hash -r 清除 shell 命令路径缓存，标准错误丢弃；不删除磁盘上的模型缓存。

L063  [空行]
空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L064      if ! [ -z "${_OLD_VIRTUAL_PS1+_}" ] ; then
检查是否保存过旧终端提示符 PS1。

L065          PS1="$_OLD_VIRTUAL_PS1"
恢复原先终端提示符字符串。

L066          export PS1
导出 PS1。

L067          unset _OLD_VIRTUAL_PS1
清掉提示符备份。

L068      fi
结束提示符恢复分支。

L069  [空行]
空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L070      unset VIRTUAL_ENV
删除 VIRTUAL_ENV 变量，表示当前不再激活某个虚拟环境。

```

```
L071      unset VIRTUAL_ENV_PROMPT
```

删除 VIRTUAL_ENV_PROMPT 环境名称变量。

```
L072      if [ ! "${1-}" = "nondestructive" ] ; then
```

除非以 nondestructive 参数调用，否则下一步连 deactivate 函数本身也删除。

```
L073      # Self destruct!
```

注释中的 Self destruct 指删除这个 shell 函数，不是删除虚拟环境文件。

```
L074      unset -f deactivate
```

unset -f deactivate 删除函数定义；项目目录和依赖仍在磁盘。

```
L075      fi
```

结束 nondestructive 判断。

```
L076 }
```

右花括号结束 deactivate 函数定义。

```
L077 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L078 # unset irrelevant variables
```

注释：正式激活前先清除旧激活留下的无关变量。

```
L079 deactivate nondestructive
```

立即调用刚定义的 deactivate，但传 nondestructive 保留函数，以便之后还能退出当前新环境。

```
L080 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L081 VIRTUAL_ENV='/home/xu/.venv'
```

设置虚拟环境完整路径 /home/xu/.venv；本环境是按当前绝对位置创建的，不是随意移动目录就能自动更新。

```
L082 if ([ "$OSTYPE" = "cygwin" ] || [ "$OSTYPE" = "msys" ]) && $(command -v cygpath &> /dev/null) ;
then
```

兼容 Cygwin/MSYS 系统路径转换的条件；本 Linux 系统通常不进入此分支，cygpath 检查仅在条件需要时执行。

```
L083     VIRTUAL_ENV=$(cygpath -u "$VIRTUAL_ENV")
```

在上述兼容环境中把路径转成 Unix 风格；当前 Linux 不需要执行。

```
L084 fi
```

结束路径转换分支。

```
L085 export VIRTUAL_ENV
```

导出 VIRTUAL_ENV，告诉后续工具当前虚拟环境的位置。

```
L086 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L087 # Unset the `SCRIPT_PATH` variable, now that the `VIRTUAL_ENV` variable
```

注释：VIRTUAL_ENV 已确定，脚本路径这个临时变量可以恢复或删除。

```
L088 # has been set. This is important for relocatable environments.
```

续行说明这对可迁移环境很重要。

```
L089 if ! [ -z "${_OLD_SCRIPT_PATH+}" ] ; then
```

检查开头是否保存了旧 SCRIPT_PATH。

```
L090     SCRIPT_PATH="_OLD_SCRIPT_PATH"
```

有旧值就恢复。

```
L091     export SCRIPT_PATH
```

导出恢复的 SCRIPT_PATH。

```
L092     unset _OLD_SCRIPT_PATH
```

删除备份变量。

```
L093 else
```

若原来没有 SCRIPT_PATH，进入另一分支。

```
L094     unset SCRIPT_PATH
```

删除本脚本临时设置的 SCRIPT_PATH。

```
L095 fi
```

结束恢复/删除分支。

```
L096 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L097 _OLD_VIRTUAL_PATH="$PATH"
```

保存当前 PATH，以便将来 deactivate 恢复。

```
L098 PATH="$VIRTUAL_ENV/bin:$PATH"
```

把 .venv/bin 放到 PATH 最前面；终端找 python 时先找到项目解释器，这是激活最关键的一步。

```
L099 export PATH
```

导出新的命令搜索路径。

```
L100 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L101 if [ "x" != x ] ; then
```

生成模板留下的固定比较 x != x 恒为假，因此当前实际总是走下面 else；通常不需手改。

```
L102     VIRTUAL_ENV_PROMPT=""
```

不执行的分支会把环境提示名设为空，用于某些模板定制情况。

```
L103 else
```

当前实际进入 else 分支。

```
L104     VIRTUAL_ENV_PROMPT=$(basename "$VIRTUAL_ENV")
```

用 basename 取 /home/xu/.venv 的最后一级 .venv，作为提示符显示名。

```
L105 fi
```

结束提示名选择分支。

```
L106 export VIRTUAL_ENV_PROMPT
```

导出 VIRTUAL_ENV_PROMPT。

```
L107 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L108 # unset PYTHONHOME if set
```

注释：若用户原来设置 PYTHONHOME，需要暂时取消，以免干扰虚拟环境。

```
L109 if ! [ -z "${PYTHONHOME+}" ] ; then
```

检查 PYTHONHOME 是否已声明。

```
L110     _OLD_VIRTUAL_PYTHONHOME="$PYTHONHOME"
```

把旧 PYTHONHOME 值备份下来。

```
L111     unset PYTHONHOME
```

unset 暂时移除它，让虚拟环境自己决定 Python 的基础路径。

```
L112 fi
```

结束 PYTHONHOME 检查。

```
L113 [空行]
```

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

```
L114 if [ -z "${VIRTUAL_ENV_DISABLE_PROMPT}" ] ; then
```

若没有显式禁止改提示符，则进入提示符设置分支。

```
L115     _OLD_VIRTUAL_PS1="${PS1-}"
```

备份原先 PS1，\${PS1-} 在变量未设置时给空字符串。

```
L116     PS1="(${VIRTUAL_ENV_PROMPT}) ${PS1-}"
```

在原提示符前面加 (.venv) 等环境名，使用户看出已激活。

```
L117     export PS1
```

导出新提示符。

```
L118 fi
```

结束提示符分支。

L119 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L120 # Make sure to unalias pydoc if it's already there

注释提醒如果 pydoc 是 alias，先取消以免影响后面定义函数。

L121 alias pydoc 2>/dev/null >/dev/null && unalias pydoc || true

查询并取消已有 pydoc alias，抑制输出和无 alias 时的失败；不删除 Python pydoc 模块。

L122 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L123 pydoc () {

定义新的 pydoc shell 函数。

L124 python -m pydoc "\$@"

调用当前 PATH 中的 python -m pydoc，并原样传递用户参数，保证查看的是当前环境的文档。

L125 }

结束 pydoc 函数。

L126 [空行]

空行。仅把相邻概念分开，Python/Bash 不会因这一行执行任何操作。

L127 # The hash command must be called to get it to forget past

注释再次解释更新 PATH 后要让 shell 忘记旧命令位置。

L128 # commands. Without forgetting past commands the \$PATH changes

续行说明不清缓存可能不遵守新 PATH。

L129 # we made may not be respected

续行结束说明。

L130 hash -r 2>/dev/null || true

清除命令路径缓存并忽略不支持时的错误，激活过程到此结束；不启动模型或 Agent。

附录 B：校验与文档维护

这份文档由当前源码与人工编写的逐行解释共同生成。程序会检查所有源代码行都有解释，并与 `reviewed_source_hashes.json` 中已审核的源码指纹比较。代码变化后会停止生成，避免把旧解释错误套到新代码上。`docs/guide/source_manifest.json` 保存交付文件的哈希。

若以后修改业务代码，需要先逐行更新 `docs/guide/make_annotations.py` 对应解释与行号，再用 `sha256sum` 对应文件 取得新哈希，更新 `docs/guide/reviewed_source_hashes.json` 的同名条目。不要只更新哈希而不检查解释。普通读者直接阅读当前 PDF 即可，不必执行这一维护操作。

重新生成命令：

```
cd /home/xu
source scripts/activate.sh
python scripts/build_guide.py
```

文档源章节在 `docs/guide/chapters.md`；逐行说明在 `docs/guide/make_annotations.py`，生成的结构化解释在 `annotations.json`；排版程序在 `scripts/build_guide.py`。旧版 PDF 保存在 `docs/USER_GUIDE_v1.pdf`。

修改示例的检查记录在 `docs/guide/example_checks.json`：只进行了语法和隔离的局部逻辑验证，没有替换正式代码、下载新模型或启动 GPU。已部署系统的历史运行证据见 `logs/verification.json`；两者不是同一类验证。

参考：Qwen3-4B 官方模型卡 <https://huggingface.co/Qwen/Qwen3-4B>；vLLM 0.8.5 API 说明 https://docs.vllm.ai/en/v0.8.5/serving/openai_compatible_server.html。具体文件顺序与项目行为以附录中的本地代码为准。